Universidad Peruana Unión

Facultad de Ingeniería y Arquitectura

Escuela Profesional de Ingeniería de Sistemas

INFORME DEL PROYECTO INTEGRADOR

ProLech: sistema web y aplicación móvil offline-first (Laravel + Kotlin Multiplatform) para el acopio de leche, el control de calidad, la liquidación a ganaderos, la planilla de choferes, el costeo real por litro, la producción y el comercio electrónico de la planta quesera ProLech (Huata, Puno)

Curso: Ingeniería de Software I

Integrantes:

Coaquira Ramírez, Jayson Smit (Developer – autor principal)

Gutiérrez Huanca, Alex Brayan (Product Owner)

Mamani Cruz, Jhon Saul (Scrum Master)

Quilla Larico, Jhon Fredy (Developer)

Docente: Mtro. Noe Wilber Tipo Mamani

Ciclo 6 · Semestre académico 2026-2

Juliaca – Perú

2026

Resumen

La planta quesera ProLech, ubicada en Huata (Puno), acopia leche fresca de ganaderos distribuidos en cuatro rutas rurales (Collana 1 a 4) y la transforma en queso tipo paria, queso andino, queso fresco, yogur, mantequilla y manjar blanco. Antes del proyecto el acopio se anotaba en cuadernos, la calidad no tenía historial, la liquidación semanal a los ganaderos y el pago a los choferes se calculaban a mano, el costo de la leche se tomaba como un valor fijo y no se sabía dónde estaba cada camión; además, en la ruta la señal de celular es intermitente. El objetivo fue desarrollar ProLech, un sistema compuesto por un backend Laravel 13 (patrón MVC con capa de servicios) con panel web por roles y tienda en línea, y una aplicación de campo offline-first construida con Kotlin Multiplatform, Compose Multiplatform, SQLDelight 2 y Ktor, que guarda cada entrega en el celular y la sincroniza con la API REST cuando vuelve la conexión, resolviendo conflictos por Last-Write-Wins. El proyecto se gestionó con Scrum en **dos sprints de cuatro semanas**: el Sprint 1 construyó el núcleo de acopio, la calidad con Lactoscan, la regla de dos adulteraciones, la rotación de choferes y la liquidación; el Sprint 2, la app offline-first con GPS, la planilla de choferes, el costo real por litro, la producción maestro–detalle y el comercio electrónico. Se entregaron 128 de 139 puntos de historia (92.1 %), con una velocidad promedio de 64 puntos por sprint, y se ejecutaron 175 pruebas automatizadas del backend (775 aserciones), 63 de la app y 87 verificaciones de extremo a extremo y de aceptación, todas aprobadas. Se concluye que el enfoque offline-first con identificadores generados en el celular elimina la dependencia de la señal sin crear duplicados, y que integrar en una sola base de datos el acopio, la calidad, los pagos y la producción permite calcular el costo real del litro puesto en planta y fijar precios con datos y no con supuestos.

Palabras clave: aplicación móvil, offline-first, Kotlin Multiplatform, Laravel, Scrum, costeo por litro

Abstract

The ProLech cheese plant in Huata (Puno, Peru) collects fresh milk from farmers along four rural routes and turns it into cheese, yogurt, butter and dulce de leche. Before this project, collection was written in notebooks, quality checks had no history, farmer settlements and driver payroll were calculated by hand, the cost of milk was a fixed assumption and nobody knew where each truck was; mobile coverage along the routes is also intermittent. The goal was to build ProLech, made of a Laravel 13 backend (MVC with a service layer) with a role-based web panel and an online store, and an offline-first field app built with Kotlin Multiplatform, Compose Multiplatform, SQLDelight 2 and Ktor, which stores every delivery on the phone and synchronizes it with the REST API when the connection returns, resolving conflicts with Last-Write-Wins. The project was managed with Scrum in **two four-week sprints**: Sprint 1 delivered milk collection, Lactoscan quality, the two-strike rule, driver rotation and farmer settlements; Sprint 2 delivered the offline-first app with GPS, driver payroll, the real cost per liter, master–detail production and e-commerce. The team delivered 128 of 139 story points (92.1 %) with an average velocity of 64 points per sprint, and ran 175 backend tests (775 assertions), 63 app tests and 87 end-to-end and acceptance checks, all passing. Phone-generated identifiers remove the dependency on coverage without duplicates, and a single database for collection, quality, payments and production makes it possible to compute the real cost of a liter delivered to the plant and to price products from data instead of assumptions.

Keywords: mobile application, offline-first, Kotlin Multiplatform, Laravel, Scrum, cost per liter

Índice

Resumen 2

Abstract 3

Índice 4

Introducción 7

Capítulo I. Descripción del Sistema y Requerimientos. 8

1.1 Situación problemática 8

1.2 Objetivos 8

1.3 Módulos del sistema. 9

1.4 Parámetros de calidad de la leche 11

1.5 Áreas del SWEBOK aplicadas 12

1.6 Requerimientos 12

Capítulo II. Gestión del Proyecto con Scrum. 14

2.1 Equipo Scrum. 14

2.2 Historias de usuario con Gherkin, INVEST y MoSCoW 14

HU-07. Regla de 2 adulteraciones por semana (Sprint 1 · Must · 8 puntos) 15

HU-08. Liquidación semanal (Sprint 1 · Must · 8 puntos) 15

HU-13 y HU-14. Acopio sin señal y sincronización (Sprint 2 · Must · 8 + 8 puntos). . . . 15

HU-22. Planilla de choferes (Sprint 2 · Must · 5 puntos) 16

HU-23. Costo real por litro y costeo dinámico (Sprint 2 · Must · 5 puntos) 16

2.3 Product Backlog priorizado con MoSCoW. 17

2.4 Planificación de los dos sprints. 19

2.5 Sprint Backlog de cada sprint. 19

2.6 Eventos Scrum 21

2.7 Métricas de los dos sprints. 21

Capítulo III. Arquitectura, Datos y Reglas del Negocio 26
3.1 Stack tecnológico 26
3.2 Backend Laravel con patrón MVC y capa de servicios 27
3.3 Modelo de datos 28
3.4 Fórmulas del negocio 29
3.4.1 Liquidación semanal del ganadero. 30
3.4.2 Regla de 2 adulteraciones y control sorpresa 30
3.4.3 Planilla de choferes 31
3.4.4 Costo real por litro puesto en planta. 31
3.4.5 Vida útil y stock por lotes 32
3.5 API REST 32
3.6 Algoritmo de sincronización offline-first 33
3.7 Arquitectura de la app KMP 34
3.8 Interfaces del sistema 35
Capítulo IV. Pruebas, Configuración y Adaptación a OpenUP 42
4.1 Resultados de pruebas 42
4.2 Evidencias de pruebas 43
4.3 Gestión de la configuración 44
4.4 Comparación con el ciclo de vida de OpenUP 45
Conclusiones. 47
Recomendaciones 48
Referencias. 49
Apéndices 50
Apéndice A. Manual de Usuario de ProLech 50
Apéndice B. Actas de los eventos Scrum. 51

B.1 Sprint 1 – Core de acopio, calidad y liquidación. 51

B.2 Sprint 2 – App offline-first y negocio 52

Apéndice C. Plantillas de Gestión de la Configuración. 54

Apéndice D. Evidencias de Despliegue 55

Introducción

El distrito de Huata, en la provincia y región de Puno, forma parte de la cuenca lechera del altiplano. La planta quesera ProLech recoge la leche de los ganaderos con cinco choferes: cuatro titulares que recorren las rutas Collana 1, 2, 3 y 4, y un volante que cubre descansos y faltas; cada mes los choferes rotan de ruta. En la planta la leche se convierte en quesos, yogures, mantequilla y manjar blanco, que se venden en mostrador y, desde este proyecto, en una tienda en línea. La semana de liquidación va del jueves al miércoles y el viernes siguiente se paga a cada ganadero según los litros recibidos, su calidad y las penalidades por adulteración.

Una solución web tradicional no basta, porque en varias zonas de la ruta no hay señal: si la app dependiera del servidor, el chofer no podría registrar la entrega en el predio. Por eso ProLech se construyó con dos piezas complementarias: un **backend Laravel** que concentra la lógica del negocio (calidad, regla de dos adulteraciones, control sorpresa, liquidación, planilla de choferes, costo real por litro, producción, ventas, tienda y respaldos) y una **app de campo offline-first** en Kotlin Multiplatform que guarda todo en SQLDelight y sincroniza en segundo plano con Ktor cuando hay conexión, sin crear duplicados.

El proyecto aplica las áreas de conocimiento del SWEBOK (Washizaki, 2024), se gestionó con Scrum (Schwaber y Sutherland, 2020) en dos sprints y se compara con OpenUP (Kroll y MacIsaac, 2006). El Capítulo I describe el problema, los módulos y los requerimientos; el Capítulo II, la gestión con Scrum: historias de usuario con Gherkin, INVEST y MoSCoW, la planificación de los dos sprints, sus eventos y métricas; el Capítulo III, la arquitectura, el modelo de datos, las fórmulas del negocio y el algoritmo de sincronización; y el Capítulo IV, las pruebas, la gestión de la configuración y la comparación con OpenUP. Los apéndices contienen el manual de usuario, las actas completas de los eventos Scrum, las plantillas de configuración y las evidencias de despliegue.

Capítulo I. Descripción del Sistema y Requerimientos

1.1 Situación problemática

Antes de ProLech, la información del acopio se perdía entre el cuaderno del chofer, la hoja de cálculo de la oficina y la memoria del personal. En las entrevistas con el administrador, la asistente administrativa y los choferes (agosto de 2026) se identificaron los siguientes problemas:

Tabla 1

Línea base del proceso manual (agosto de 2026)

Indicador	Situación antes de ProLech	Meta con ProLech
Registro del acopio en la ruta	Cuaderno del chofer, transcrito en la planta al día siguiente	Registro en el celular en el predio, con o sin señal
Acopios duplicados o con errores de transcripción	Frecuentes (estimado 5 %)	0 duplicados (UUID por acopio)
Tiempo para calcular la liquidación semanal	1 a 2 días de trabajo de la asistente	Menos de 1 minuto
Pago a los choferes	Jornal anotado a mano, sin bono ni km	Planilla automática por días, litros y km con boleta PDF
Costo de la leche en las recetas	Valor fijo (S/ 1.80) que no cambiaba	Costo real por litro puesto en planta, recalculado cada día
Detección de la adulteración	Al cierre de la semana, si alguien revisaba	Inmediata y con controles sorpresa sorteados cada día
Ubicación de los camiones	Llamadas telefónicas al chofer	Mapa en vivo y recorrido con km
Respaldo de la información	Ninguno	Respaldo automático diario de la base de datos

Nota. Valores de la situación inicial estimados por el equipo a partir de entrevistas al personal de la planta (caso de estudio).

1.2 Objetivos

Objetivo general. Desarrollar ProLech, un sistema web y móvil con enfoque offline-first y backend Laravel que integre el acopio de leche, el control de calidad, la liquidación a ganaderos, la planilla de choferes, el costeo real por litro, la producción, el inventario y las ventas de la planta quesera ProLech, para eliminar la dependencia de la señal en la ruta y contar con información confiable para pagar, producir y fijar precios.

Objetivos específicos.

- Registrar en el celular, con o sin señal, los litros de cada ganadero, una observación a simple vista y el frasco de muestra cuando la planta lo pide.
- Clasificar la calidad (aprobado, observado o rechazado), suspender al ganadero con dos adulteraciones en la semana y sortear controles sorpresa con sanciones graduales.
- Generar la liquidación semanal de cada ganadero (jueves a miércoles) con precio por litro, penalidades y descuentos.
- Calcular la planilla de los choferes (días con ruta, litros recolectados, km recorridos, bono y descuentos) y emitir su boleta en PDF y Excel.
- Calcular el costo real por litro puesto en planta (precio al ganadero + logística + gastos generales) y usarlo en las recetas y en el precio sugerido de venta.
- Controlar las órdenes de producción (maestro–detalle), los insumos, la vida útil de cada lote y el stock de la tienda.
- Vender en mostrador y en la tienda web con comprobante, boleta y seguimiento del pedido.

1.3 Módulos del sistema

Tabla 2

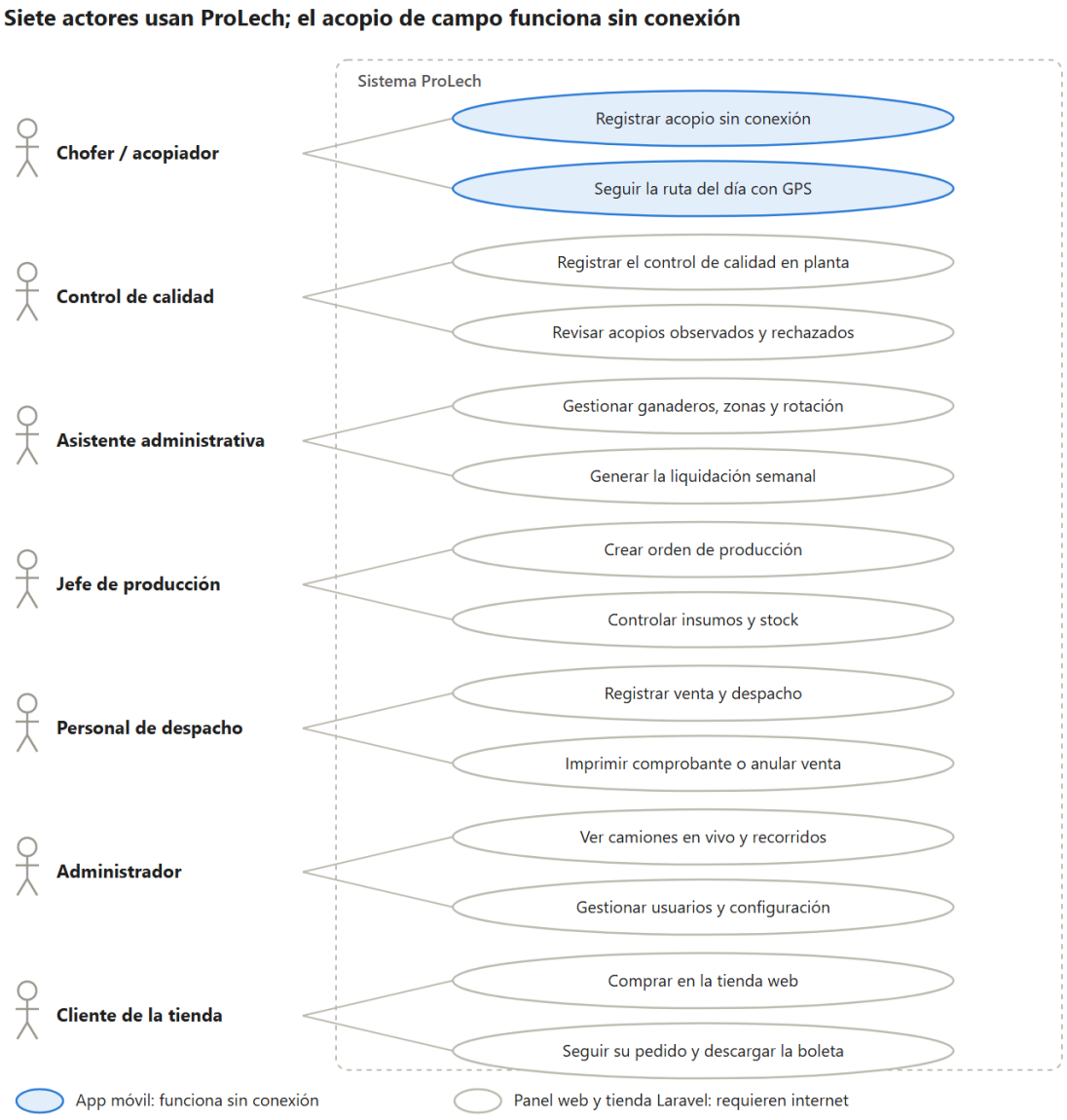
Módulos de ProLech, usuarios y plataforma

Módulo	Usuario principal	Plataforma	Sin conexión
Acopio en campo	Chofer	App KMP «ProLech Acopio»	Sí
Rutas, GPS y trazado	Chofer; administrador	App (OSMDroid) y panel web	Sí (GPS en lote)
Control de calidad y control sorpresa	Calidad; administrador	Panel web; frasco en la app	Parcial
Ganaderos, rutas y rotación	Asistente; administrador	Panel web	No
Liquidación semanal	Asistente; administrador	Panel web (PDF y Excel)	No
Planilla de choferes	Administrador	Panel web (boleta PDF y Excel)	No
Costo real por litro	Administrador; jefe de producción	Panel web (tarjeta en recetas, producción y dashboard)	No
Producción, recetas e inventario	Jefe de producción	Panel web	No
Ventas, despacho y tienda en línea	Despacho; cliente	Panel web y tienda pública	No
Operación y reportes	Administrador	Panel web y tareas programadas	No

Nota. Elaboración propia.

Figura 1

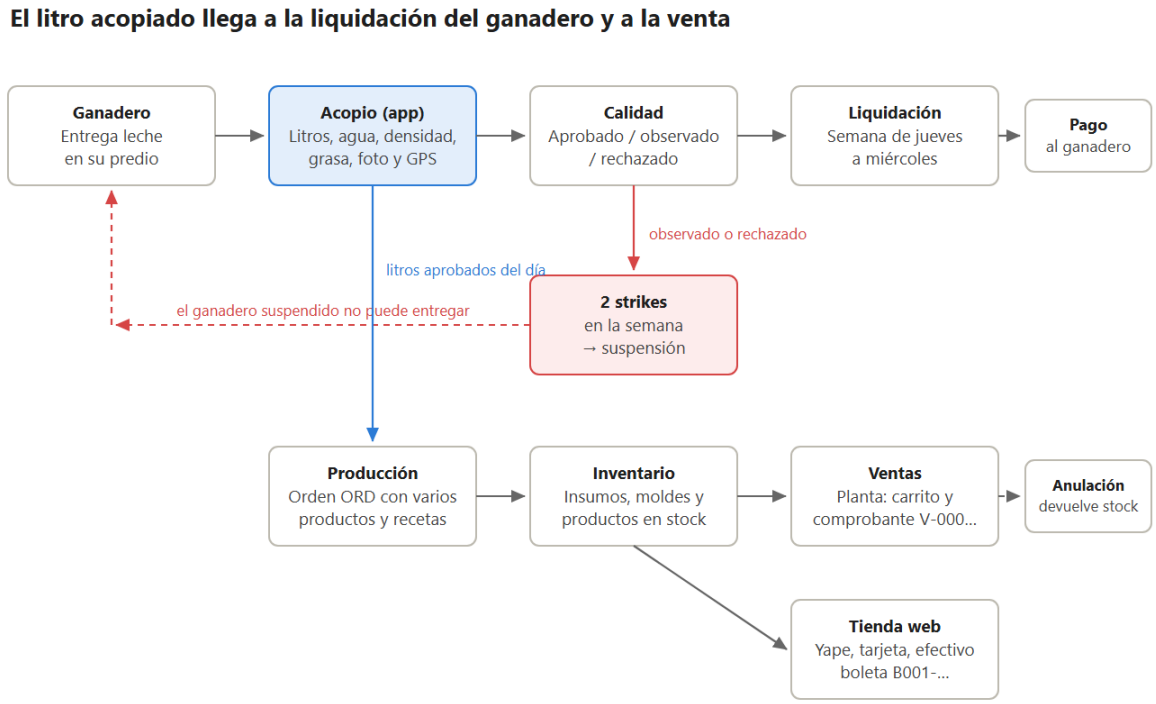
Diagrama de casos de uso de ProLech



Nota. Los casos resaltados se ejecutan en la app de campo y se sincronizan automáticamente; los demás se realizan en el panel web o en la tienda. Cada rol del panel ve solo sus secciones (clase `RolWeb`) y un usuario puede tener varios roles. Elaboración propia.

Figura 2

Flujo de información entre módulos



Nota. El litro acopiado alimenta la liquidación del ganadero, la planilla del chofer (litros y km) y el costo real por litro que usan las recetas para el precio sugerido de venta. Elaboración propia.

1.4 Parámetros de calidad de la leche

Desde el Sprint 2 el chofer ya no mide la calidad: registra los litros y una observación a simple vista (normal, olor, color, grumos, muy caliente u otro). La calidad la miden el Lactoscan de la planta y los controles sorpresa del personal de calidad. Los parámetros se configuran en el panel (tabla `configuracion`):

Tabla 3

Parámetros de aceptación y efectos en el pago

Parámetro	Rango o condición	Efecto en el sistema
Agua agregada	> 5 %(grave)	Toda la semana se paga a S/ 0.65 por litro
Agua agregada	≤ 5 %con estado observado o rechazado (leve)	1.ª vez: semana a P − S/ 0.10; 2.ª vez: se retiran esas entregas
Agua agregada	≈ 1 %(descuento puntual)	Esas entregas a P − S/ 0.10
pH (acidez)	Fuera de 6.4 a 6.8 / menor que 5.5 o mayor que 8.5	Observado / rechazado

Parámetro	Rango o condición	Efecto en el sistema
Densidad	Fuera de 1.028 a 1.033 g/ml	Observado
Adulteraciones en la semana	2 o más (jueves a miércoles)	Ganadero suspendido y aviso al administrador
Control sorpresa grave	Frasco o predio con agua > 5 %	Esa leche no se paga y se bloquean 3 entregas (6 si reincide; a la 3.ª, suspensión)
Control sorpresa leve	Agua > 0 % y ≤ 5 %	Descuento en esa entrega y aviso al ganadero
Precio por litro (P)	Temporada normal / lluvias	S/ 1.70 / S/ 1.20

Nota. Los rangos deben validarse con el laboratorio de la planta y la normativa sanitaria vigente. Elaboración propia a partir de `LiquidacionController::calcularLiquidacion` y `ControlCalidadService`.

1.5 Áreas del SWEBOK aplicadas

Tabla 4

Áreas de conocimiento del SWEBOK aplicadas en ProLech

Área SWEBOK	Aplicación en ProLech
Requerimientos	Historias de usuario con criterios Gherkin (Dado/Cuando/Entonces), evaluadas con INVEST y priorizadas con MoSCoW
Arquitectura y diseño	Laravel MVC con capa de servicios; arquitectura limpia en la app (presentación, dominio, datos) con SQLDelight como fuente única de verdad
Construcción	PHP 8.5 y Kotlin 2.4; Koin para inyección de dependencias; 67 migraciones versionadas en el servidor y 4 en el celular
Pruebas	PHPUnit (175 pruebas), kotlin.test (63 pruebas), pruebas de extremo a extremo y en emulador
Gestión de la configuración	Git en dos repositorios, convención de commits, secretos fuera del repositorio, CI con GitHub Actions
Economía de la ingeniería de software	Costo real por litro y precio sugerido calculados con datos del propio sistema
Operaciones y mantenimiento	Docker + Caddy con HTTPS, respaldos diarios y 9 tareas programadas
Seguridad	Token Sanctum, verificación en dos pasos por WhatsApp, permisos por rol y sección, auditoría

Nota. Adaptado de Washizaki (2024).

1.6 Requerimientos

Tabla 5

Requerimientos funcionales y no funcionales

ID	Requerimiento	Módulo	Sprint
RF-01	Iniciar sesión por rol (web con verificación en dos pasos; app con token) y asignar varios roles a un usuario	Seguridad	1
RF-02	Gestionar ganaderos, rutas con polígono y rotación mensual de choferes (titular y volante)	Acopio	1

ID	Requerimiento	Módulo	Sprint
RF-03	Registrar acopio con parámetros del Lactoscan y clasificar su calidad	Calidad	1
RF-04	Suspender al ganadero con 2 adulteraciones en la semana	Calidad	1
RF-05	Generar la liquidación semanal por ganadero y su sobre de pago en PDF	Liquidación	1
RF-06	Registrar acopios sin conexión y sincronizarlos solos al volver la señal	App KMP	2
RF-07	Mostrar la ruta del día en el mapa, enviar el GPS y avisar cambios de ruta	Rutas	2
RF-08	Sortear controles sorpresa, registrar resultados y aplicar sanciones	Calidad	2
RF-09	Calcular y pagar la planilla de choferes con boleta PDF y exportación Excel	Planilla	2
RF-10	Calcular el costo real por litro y usarlo en recetas y precios sugeridos	Costos	2
RF-11	Registrar órdenes de producción multiproducto con vencimiento por lote	Producción	2
RF-12	Vender en mostrador y en la tienda web con boleta y seguimiento	Ventas	2
RF-13	Respaldar la base de datos cada día y conservar los últimos 14 respaldos	Operación	2
RNF-01	Toda operación de la app funciona en modo avión	Móvil	2
RNF-02	La sincronización no duplica registros (UUID generado en el celular)	Sincronización	2
RNF-03	Ante una edición en dos equipos gana el cambio más reciente (Last-Write-Wins)	Sincronización	2
RNF-04	HTTPS en producción y autenticación por token Bearer	Seguridad	1
RNF-05	Máximo 5 intentos de inicio de sesión por minuto	Seguridad	1
RNF-06	La app funciona en Android 7.0 (API 24) o superior	Móvil	2
RNF-07	Fechas en la zona horaria de Lima; montos con 2 decimales y redondeo a S/ 0.50 en precios	General	1

Nota. RF = requerimiento funcional; RNF = requerimiento no funcional. Elaboración propia.

Capítulo II. Gestión del Proyecto con Scrum

2.1 Equipo Scrum

Tabla 6

Roles del equipo Scrum

Integrante	Rol Scrum	Responsabilidades en ProLech
Jayson Smit Coaquira Ramírez	Developer (autor principal)	Arquitectura del sistema; backend Laravel (modelos, servicios, API REST, liquidación, control sorpresa, planilla y costo real por litro); app Kotlin Multiplatform (SQLDelight, Ktor, SyncManager, OSMDroid); pruebas automatizadas y redacción del informe
Alex Brayan Gutiérrez Huanca	Product Owner	Representa al administrador de la planta; redacta y prioriza el Product Backlog con MoSCoW; define criterios de aceptación y acepta los incrementos en la Sprint Review
Jhon Saul Mamani Cruz	Scrum Master	Facilita los eventos y elimina impedimentos; lleva el tablero, el burndown y las métricas; diseña y ejecuta los casos de prueba de aceptación
Jhon Fredy Quilla Larico	Developer	Vistas Blade del panel y la tienda, reportes y exportaciones, pruebas de extremo a extremo, despliegue con Docker y respaldos

Nota. Equipo de cuatro integrantes. Elaboración propia.

Definición de Listo (DoR). Una historia entra al sprint cuando: (a) está escrita como «Como … quiero … para …»; (b) tiene al menos dos escenarios Gherkin; (c) cumple INVEST; (d) está estimada en puntos Fibonacci por todo el equipo (Planning Poker); y (e) el Product Owner le asignó prioridad MoSCoW

Definición de Hecho (DoD). Una historia está terminada cuando: (a) cumple todos sus escenarios Gherkin; (b) tiene pruebas automatizadas que pasan (`php artisan test` o `gradlew testAndroidHostTest`); (c) la migración corre en MySQL y en SQLite; (d) la parte móvil funciona en modo avión y sincroniza al volver la señal; (e) el panel no muestra errores (barrido de páginas en Chrome sin errores de JavaScript); y (f) el Product Owner la aceptó en la Sprint Review

2.2 Historias de usuario con Gherkin, INVEST y MoSCoW

Las historias usan el formato «Como [rol], quiero [acción], para [beneficio]». Los criterios de aceptación se escribieron en Gherkin (Dado/Cuando/Entonces) para que el Product Owner los lea y el equipo los convierta directamente en pruebas automatizadas; cada escenario de esta sección

tiene una prueba con el mismo nombre en el repositorio. A continuación se detallan las historias de mayor valor de cada sprint.

HU-07. Regla de 2 adulteraciones por semana (Sprint 1 · Must · 8 puntos)

> **Como** administrador de la planta, **quiero** que el sistema suspenda automáticamente al ganadero que adultera dos veces en la misma semana de liquidación, **para** proteger la calidad del queso sin depender de que alguien revise los cuadernos.

```
Característica: Regla de 2 strikes por semana de liquidación (jueves a miércoles)

  Escenario: Segunda adulteración en la misma semana suspende al ganadero
    Dado que «Rosa Quispe» tiene un acopio OBSERVADO el jueves 15/10/2026
    Cuando el chofer registra otro acopio OBSERVADO de ella el lunes 19/10/2026
    Entonces el ganadero queda en estado «Suspendido»
    Y el administrador recibe la notificación «adulteracion_exceso»

  Escenario: El reenvío del mismo acopio no cuenta doble
    Dado un acopio OBSERVADO con uuid «7f3c…»
    Cuando el celular reenvía el mismo uuid por falta de respuesta
    Entonces el ganadero sigue «Activo» y solo hay 1 notificación
```

HU-08. Liquidación semanal (Sprint 1 · Must · 8 puntos)

> **Como** asistente administrativa, **quiero** generar con un clic la liquidación de todos los ganaderos de la semana, **para** pagar el viernes sin pasar dos días haciendo cálculos.

```
Escenario: Semana con adulteración grave se paga a precio de penalidad
  Dado que el ganadero entregó 300.90 L entre el jueves y el miércoles
  Y uno de sus acopios tuvo 7 % de agua (más del 5 %)
  Cuando genero las liquidaciones de esa semana
  Entonces su liquidación es 300.90 L × S/ 0.65 = S/ 195.59
  Y el detalle dice «Adulteración >5%: TODA la semana a S/ 0.65/L»

Escenario: La leche de una muestra sorpresa grave no se paga
  Dado que entregó 100 L buenos y 40 L con muestra sorpresa grave
  Cuando genero la liquidación
  Entonces se pagan 100 L y el detalle dice «40 L no se pagan»
```

HU-13 y HU-14. Acopio sin señal y sincronización (Sprint 2 · Must · 8 + 8 puntos)

> **Como** chofer, **quiero** registrar los litros de cada ganadero aunque no haya señal y que se suban solos al volver la red, **para** no perder ni duplicar entregas.

```
Escenario: Registrar en modo avión
  Dado que el celular está en modo avión
  Cuando registro 40 L de «Juan Quispe» con observación «normal»
  Entonces el acopio se guarda en SQLDelight con estadoSync PENDIENTE_CREAR
  Y la lista lo muestra con el aviso «Pendiente de subir»

Escenario: Sincronización automática al recuperar la red
  Dado que hay 3 acopios pendientes
  Cuando el MonitorConectividad publica enLinea = true
  Entonces el SyncManager los envía por POST /api/acopios-campo
  Y los tres pasan a «Sincronizado» sin que el chofer toque nada

Escenario: Conflicto entre dos equipos (Last-Write-Wins)
  Dado que el mismo acopio se editó en dos celulares
  Cuando ambos sincronizan
  Entonces gana la versión con el actualizado_en más reciente
```

HU-22. Planilla de choferes (Sprint 2 · Must · 5 puntos)

> **Como** administrador, **quiero** que la planilla de cada chofer se calcule sola con sus días de ruta, litros y kilómetros, **para** pagarle lo justo, entregarle su boleta y conocer el costo de la logística.

```
Escenario: Cálculo de la planilla semanal
  Dado un jornal de S/ 60.00, un bono de S/ 0.02 por litro y pensión de 13 %
  Y que «Carlos Ramos» trabajó 6 días y recogió 898.7 L entre el 22/10 y el 28/10
  Cuando genero la planilla semanal
  Entonces su bruto es 6 × 60 + 898.7 × 0.02 = S/ 377.97
  Y su neto es 377.97 − 13 % = S/ 328.83

Escenario: Confirmar el pago y descargar la boleta
  Dado una planilla en estado «Por pagar»
  Cuando confirmo el pago por Yape con la operación «OP-4121»
  Entonces queda «Pagada», ya no se recalcula y puedo descargar la boleta PDF
```

HU-23. Costo real por litro y costeo dinámico (Sprint 2 · Must · 5 puntos)

> **Como** jefe de producción, **quiero** que el costo de la «Leche Cruda» en mis recetas sea el costo real del litro puesto en planta, **para** que el precio sugerido de cada queso y yogur cubra lo que realmente cuesta producirlo.

```
Escenario: Desglose del costo real por litro
  Dado que en 30 días se acopiaron 1 000 L pagados a S/ 1.80 en promedio
  Y la logística costó S/ 40 de combustible + S/ 140 de choferes
  Y los gastos generales de planta son 4 %
  Cuando se calcula el costo real
  Entonces es 1.80 + 0.18 + 0.0792 = S/ 2.0592 por litro

Escenario: La receta usa el costo dinámico
  Dado una receta de queso fresco con 10 L de «Leche Cruda» por unidad
  Cuando abro Recetas
  Entonces su costo unitario es 10 × 2.0592 = S/ 20.59 (no 10 × costo fijo)
```

La Tabla 7 evalúa estas historias con el criterio INVEST (Independiente, Negociable, Valiosa, Estimable, Pequeña y Testeable). Las historias que no pasaban el criterio «Pequeña» en el refinamiento se dividieron: por ejemplo, la «app offline» original se partió en HU-13 (guardar sin señal) y HU-14 (sincronizar).

Tabla 7

Evaluación INVEST de las historias principales

Historia	I	N	V	E	S	T	Observación del refinamiento
HU-07	✓	✓	✓	✓	✓	✓	Depende solo del acopio (HU-05); 8 pruebas en `Regla2StrikesTest`
HU-08	✓	✓	✓	✓	✓	✓	Se separó el sobre de pago PDF como tarea de la misma historia
HU-13	✓	✓	✓	✓	✓	✓	Dividida de la épica «app offline» para que entre en un sprint
HU-14	✓	✓	✓	✓	✓	✓	12 pruebas en `SyncManagerTest`
HU-21	✓	✓	✓	✓	✓	✓	Se negoció con el PO: el chofer ya no mide, solo observa
HU-22	✓	✓	✓	✓	✓	✓	Tarifas configurables: el PO puede cambiarlas sin código
HU-23	–	✓	✓	✓	✓	✓	Depende de HU-08 y HU-22 (precio pagado y pago a choferes)

Nota. ✓ = cumple; – = cumple parcialmente. Elaboración propia.

2.3 Product Backlog priorizado con MoSCoW

El Product Owner priorizó el backlog con MoSCoW: **Must** (sin esto no hay producto), **Should** (importante, pero el producto funciona sin ello), **Could** (deseable si sobra capacidad) y **Won't** (acordado que no entra en este release). Las estimaciones se hicieron con Planning Poker en escala Fibonacci.

Tabla 8

Product Backlog (puntos de historia en escala Fibonacci)

ID	Historia de usuario	Módulo	MoSCoW	Pts.	Sprint	Estado
HU-01	Iniciar sesión por rol en la web (2FA) y en la app (token)	Seguridad	Must	5	1	Hecho
HU-02	Usuarios con varios roles y edición de sus datos	Seguridad	Should	3	1	Hecho
HU-03	Gestionar ganaderos y rutas con polígono en el mapa	Acopio (web)	Must	5	1	Hecho
HU-04	Rotación mensual de choferes por ruta (titular y volante)	Rutas	Must	5	1	Hecho
HU-05	Registrar acopio con parámetros del Lactoscan (API)	Acopio	Must	8	1	Hecho
HU-06	Clasificar la calidad y aplicar el descuento por agua	Calidad	Must	5	1	Hecho

ID	Historia de usuario	Módulo	MoSCoW	Pts.	Sprint	Estado
HU-07	Regla de 2 adulteraciones por semana y suspensión	Calidad	Must	8	1	Hecho
HU-08	Liquidación semanal (jueves a miércoles) con sobre de pago	Liquidación	Must	8	1	Hecho
HU-09	Plan de calidad, control en planta y capacitaciones ATOS	Calidad (web)	Should	5	1	Hecho
HU-10	Reportes y exportación a Excel, PDF y CSV	Reportes	Should	5	1	Hecho
HU-11	Comunicados, reuniones y reclamos de litraje	Comunicación	Could	3	1	Hecho
HU-12	Portal del proveedor (ganadero)	Portal	Could	3	1 →2	Hecho
HU-13	App KMP: registrar acopio sin señal en SQLDelight	App KMP	Must	8	2	Hecho
HU-14	Sincronización automática push/pull con Ktor y Last-Write-Wins	App KMP	Must	8	2	Hecho
HU-15	Ruta del día con mapa OSMDroid, camino OSRM y GPS en lote	Rutas (app)	Must	5	2	Hecho
HU-16	Panel en vivo, recorridos con km y trazado de rutas	Operaciones	Should	3	2	Hecho
HU-17	Producción maestro–detalle: órdenes multiproducto, recetas e insumos	Producción	Must	8	2	Hecho
HU-18	Vencimiento por lote y stock de la tienda desde lo producido	Producción	Should	3	2	Hecho
HU-19	Ventas con carrito, comprobante, anulación y dashboard	Ventas	Must	5	2	Hecho
HU-20	E-commerce: catálogo, checkout (Yape, tarjeta, efectivo), boleta y seguimiento	Tienda	Must	8	2	Hecho
HU-21	Control de calidad sorpresa: sorteo, frasco, semáforo y sanciones	Calidad	Should	5	2	Hecho
HU-22	Planilla de choferes con boleta PDF y Excel	Planilla	Must	5	2	Hecho
HU-23	Costo real por litro y costeo dinámico de recetas	Costos	Must	5	2	Hecho
HU-24	Respaldos automáticos y tareas programadas	Operación	Must	2	2	Hecho
HU-25	Publicar el sistema en un VPS con dominio y HTTPS	Despliegue	Should	5	2	Pendiente
HU-26	Boleta con validez SUNAT (cuenta Nubefact)	Tienda	Could	3	2	Pendiente
HU-27	Pestaña «Inspecciones» en la app para el inspector	App KMP	Could	3	2	Pendiente
HU-28	App para iOS	App KMP	Won't	—	—	Fuera del release
HU-29	Pasarela de pago con tarjeta en producción	Tienda	Won't	—	—	Fuera del release

Nota. Alcance del release: 139 puntos (HU-01 a HU-27). Entregados: 128 (92.1 %). HU-12 se comprometió en el Sprint 1 y se terminó en el Sprint 2. HU-25 a HU-27 (11 puntos) quedaron pendientes; HU-28 y HU-29 se acordaron como Won't para este release.

Tabla 9

Distribución del backlog por prioridad MoSCoW

Prioridad	Historias	Puntos	Entregados	% del alcance
Must	16	98	98	70.5 %
Should	7	29	24	20.9 %
Could	4	12	6	8.6 %
Won't (este release)	2	—	—	—
Total	**29**	**139**	**128**	**100 %**

Nota. Todas las historias Must se entregaron; lo pendiente es Should o Could y depende de contratar servicios externos.

2.4 Planificación de los dos sprints

Tabla 10

Planificación de sprints y objetivo de cada sprint

Sprint	Fechas (2026)	Objetivo del sprint (Sprint Goal)	Compromiso
1	03/09 al 30/09 (4 semanas, 20 días hábiles)	Core de acopio: el chofer registra cada entrega con su calidad Lactoscan, la planta castiga la adulteración con la regla de 2 strikes, rota a sus choferes y liquida a sus ganaderos cada semana	HU-01 a HU-12: 63 puntos
2	01/10 al 28/10 (4 semanas, 20 días hábiles)	Offline y negocio: el chofer trabaja sin señal con la app KMP y GPS; la planta paga la planilla de choferes, conoce el costo real de su litro, produce con recetas maestro–detalle y vende en línea	HU-12 (arrastre) y HU-13 a HU-27: 79 puntos

Nota. Sprints de cuatro semanas (duración máxima permitida por la Guía Scrum). La Sprint Review del Sprint 1 fue el 30/09/2026 y la del Sprint 2, el 28/10/2026. Elaboración propia.

Capacidad. Cada integrante dedicó en promedio 3 h por día hábil: $4 \times 20 \times 3 = 240$ h por sprint. En el Sprint 1 el equipo no tenía velocidad histórica y comprometió 63 puntos (≈ 3.8 h por punto); en el Sprint 2 usó la velocidad del Sprint 1 (60 puntos) más 19 puntos de historias Should/Could con dependencias externas, aceptando el riesgo de no terminarlas.

2.5 Sprint Backlog de cada sprint

Tabla 11

Sprint Backlog del Sprint 1 (Core de acopio y liquidación)

Historia	Tarea	Responsable	Horas	Estado
HU-01	Login web con 2FA por WhatsApp (Call Me Bot) y login API con Sanctum	Jayson Smit Coaquira Ramírez	10	Hecho
HU-02	Roles adicionales (`roles_extra`) y middleware `rol`/`seccion`	Jhon Fredy Quilla Larico	6	Hecho

Historia	Tarea	Responsable	Horas	Estado
HU-03	CRUD de ganaderos; zonas con polígono en Leaflet	Jhon Fredy Quilla Larico	12	Hecho
HU-04	`AsignacionZona::asignar` con intercambio y volante; rotación automática	Jayson Smit Coaquira Ramírez	10	Hecho
HU-05	API `POST /acopios` idempotente con `client_uuid`; campos Lactoscan	Jayson Smit Coaquira Ramírez	14	Hecho
HU-06	`Acopio::calcularEstado` y descuento por agua	Jayson Smit Coaquira Ramírez	8	Hecho
HU-07	`AdulteracionService` y `rangoSemanaLiquidacion` (jueves–miércoles)	Jayson Smit Coaquira Ramírez	12	Hecho
HU-08	`calcularLiquidacion`, sobre de pago PDF (DomPDF) y estados	Jayson Smit Coaquira Ramírez	16	Hecho
HU-09	Plan inopinado, control en planta y capacitaciones ATOS	Jhon Fredy Quilla Larico	12	Hecho
HU-10	Reportes y generador XLSX propio	Jhon Fredy Quilla Larico	10	Hecho
HU-11	Comunicados, reuniones y reclamos de litraje	Jhon Fredy Quilla Larico	8	Hecho
HU-12	Portal del proveedor	Jhon Fredy Quilla Larico	6	Arrastrada
Todas	Casos de prueba de aceptación y tablero	Jhon Saul Mamani Cruz	20	Hecho
Todas	Refinamiento, criterios Gherkin y validación con la planta	Alex Brayan Gutiérrez Huanca	16	Hecho

Nota. Horas estimadas en el Sprint Planning del 03/09/2026. Elaboración propia.

Tabla 12

Sprint Backlog del Sprint 2 (App offline-first y negocio)

Historia	Tarea	Responsable	Horas	Estado
HU-12	Portal del proveedor (arrastre del Sprint 1)	Jhon Fredy Quilla Larico	6	Hecho
HU-13	Esquema SQLDelight 2 (`AcopioEntity`, migraciones 1.sqm–4.sqm) y repositorio offline	Jayson Smit Coaquira Ramírez	14	Hecho
HU-14	`SyncManager` push/pull con Ktor, `MonitorConectividad` y Last-Write-Wins	Jayson Smit Coaquira Ramírez	16	Hecho
HU-14	API `/api/acopios-campo` con upsert por UUID y `AcopioCampoEnlace`	Jayson Smit Coaquira Ramírez	8	Hecho
HU-15	Mapa OSMDroid, camino OSRM, GPS en lote y aviso de cambio de ruta	Jayson Smit Coaquira Ramírez	12	Hecho
HU-16	Panel en vivo, recorridos con km y trazado de rutas	Jhon Fredy Quilla Larico	8	Hecho
HU-17	Órdenes multiproducto (`ProduccionService`), recetas y movimientos	Jayson Smit Coaquira Ramírez	14	Hecho
HU-18	Vida útil, `fecha_vencimiento` y `StockLotes` (FIFO)	Jayson Smit Coaquira Ramírez	6	Hecho
HU-19	Carrito, comprobante, anulación y dashboard de ventas (Chart.js)	Jhon Fredy Quilla Larico	10	Hecho
HU-20	Tienda, checkout por pasos, boleta B001 y seguimiento	Jhon Fredy Quilla Larico	14	Hecho

Historia	Tarea	Responsable	Horas	Estado
HU-21	`ControlCalidadService`: sorteo, frasco, semáforo y sanciones	Jayson Smit Coaquira Ramírez	12	Hecho
HU-22	`PlanillaChoferService`, vistas, boleta PDF y exportación	Jayson Smit Coaquira Ramírez	10	Hecho
HU-23	`CostoLecheService`, `Insumo::costoVigente` y tarjeta de costos	Jayson Smit Coaquira Ramírez	8	Hecho
HU-24	Comando `prolech:respaldo` y scheduler (9 tareas)	Jhon Fredy Quilla Larico	4	Hecho
HU-25	Contratar VPS y dominio; publicar con Docker + Caddy	Jhon Fredy Quilla Larico	6	Pendiente
HU-26	Integración Nubefact con credenciales reales	Jhon Fredy Quilla Larico	4	Pendiente
HU-27	Pestaña «Inspecciones» del inspector en la app	Jayson Smit Coaquira Ramírez	6	Pendiente
Todas	Pruebas E2E (70 peticiones), emulador Android 14 y celular real	Jhon Saul Mamani Cruz	24	Hecho
Todas	Validación de fórmulas de planilla y costos con la planta	Alex Brayan Gutiérrez Huanca	12	Hecho

Nota. Horas estimadas en el Sprint Planning del 01/10/2026. Elaboración propia.

2.6 Eventos Scrum

Tabla 13

Eventos Scrum realizados y evidencias generadas

Evento	Frecuencia y duración	Sprint 1	Sprint 2	Evidencia
Sprint Planning	Primer día del sprint, máx. 8 h	03/09 · 4 h	01/10 · 4 h	Acta con Sprint Goal y Sprint Backlog (Apéndice B)
Daily Scrum	Cada día hábil, 15 min	20 dailies	20 dailies	Registro de avances e impedimentos
Refinamiento del backlog	Semanal, 1 h	4 sesiones	4 sesiones	Historias divididas, Gherkin e INVEST
Sprint Review	Último día, máx. 4 h	30/09 · 2 h	28/10 · 2 h	Acta de aceptación y demostración en el celular
Sprint Retrospective	Después de la Review, máx. 3 h	30/09 · 1.5 h	28/10 · 1.5 h	Empezar–Dejar–Continuar con responsables

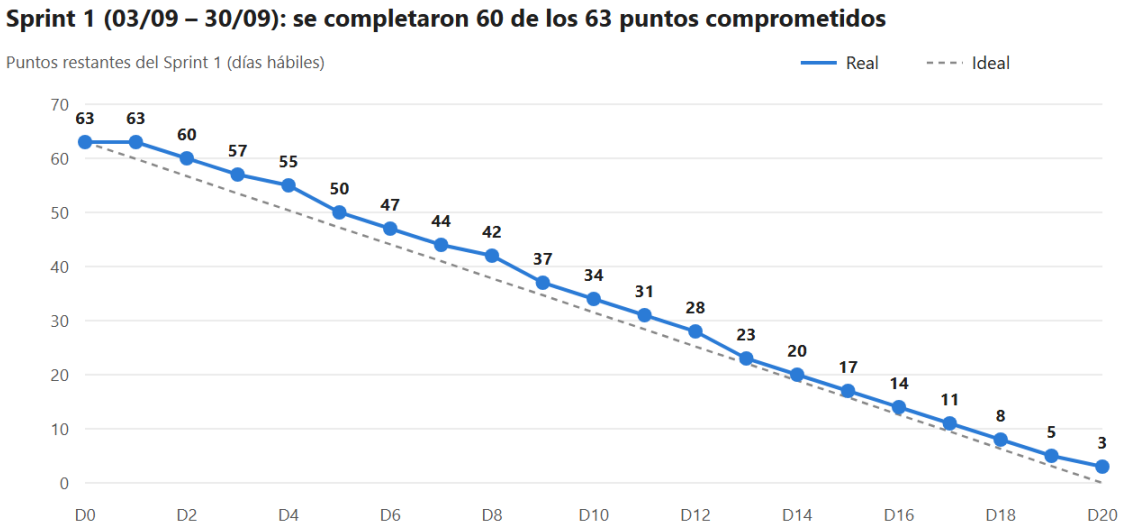
Nota. Duraciones máximas según Schwaber y Sutherland (2020) para sprints de un mes.

2.7 Métricas de los dos sprints

Las métricas de puntos se tomaron del tablero al cierre de cada día; las de calidad del producto, de la ejecución de las pruebas y del historial de Git.

Figura 3

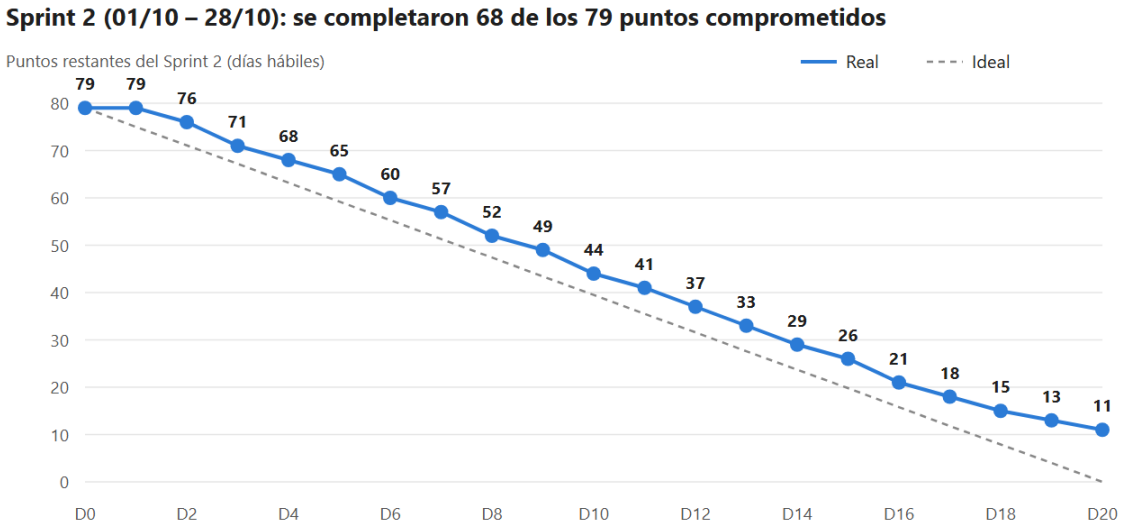
Burndown del Sprint 1: puntos restantes por día hábil



Nota. La meseta del día 1 corresponde a la configuración de Laravel 13, Sanctum y el 2FA. Los 3 puntos restantes son HU-12 (portal del proveedor), que pasó al Sprint 2.

Figura 4

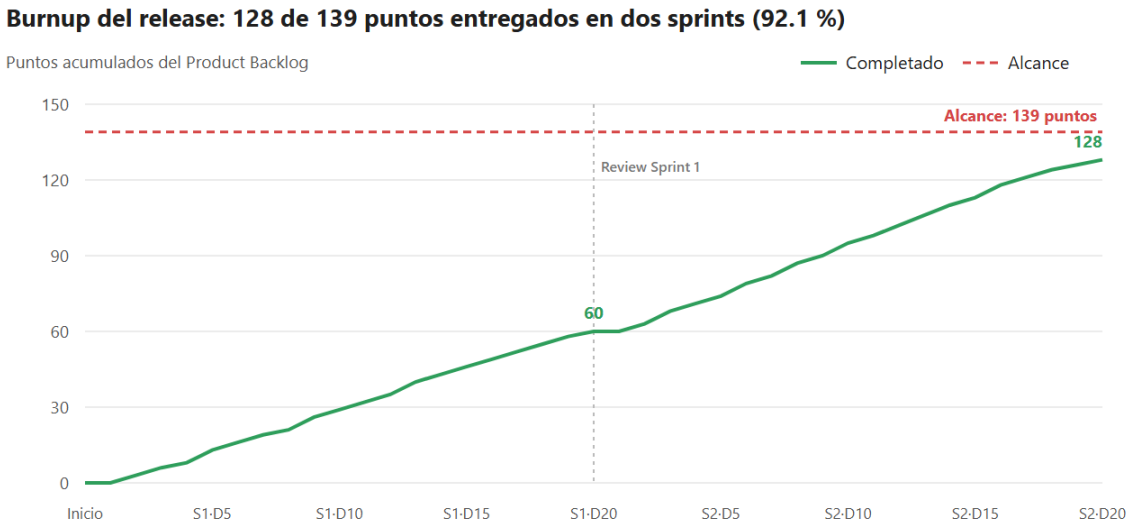
Burndown del Sprint 2: puntos restantes por día hábil



Nota. La caída más lenta de los días 1 a 4 corresponde a configurar Kotlin Multiplatform con AGP 9 y SQLDelight. Los 11 puntos restantes son HU-25 a HU-27, que dependen de contratar servicios externos o pasan al siguiente release.

Figura 5

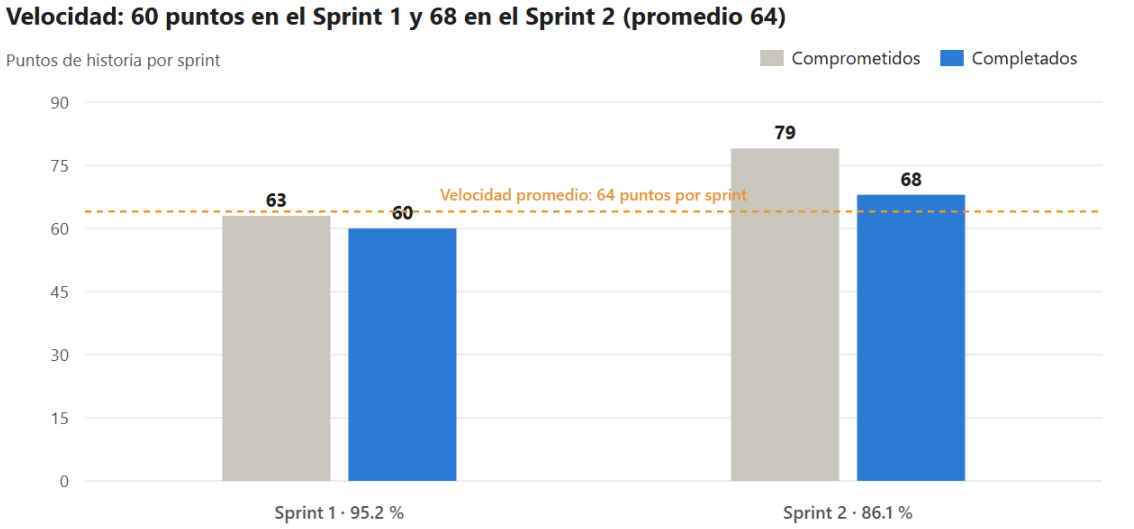
Burnup del release: alcance y puntos completados



Nota. El alcance se mantuvo en 139 puntos durante los dos sprints (no hubo cambios de alcance aprobados por el PO).

Figura 6

Velocidad del equipo: puntos comprometidos y completados



Nota. Velocidad del Sprint 1: 60 puntos (95.2 % de cumplimiento); del Sprint 2: 68 puntos (86.1 %).

Tabla 14

Indicadores del proceso de desarrollo

Indicador	Cálculo	Sprint 1	Sprint 2	Total	Meta
Velocidad	Puntos completados	60	68	Promedio 64	—
Cumplimiento del compromiso	Completados / comprometidos	60/63 = 95.2 %	68/79 = 86.1 %	128/142 = 90.1 %	≥ 85 %
Avance del producto	Entregados / alcance	43.2 %	92.1 %	128/139	≥ 85 %
Pruebas del backend	Aprobadas / ejecutadas	—	—	175 / 175 (775 aserciones)	100 %
Pruebas de la app	Aprobadas / ejecutadas	—	—	63 / 63	100 %
Defectos corregidos	Corregidos / encontrados	—	—	43 / 43	100 %
Densidad de defectos	Defectos / KLOC	—	—	43 / 55.9 = 0.77	≤ 1

Nota. Las pruebas y los defectos se miden sobre el producto al cierre del Sprint 2 (28/10/2026): 35 defectos de la revisión integral del 25/10 y 8 de la última semana, todos corregidos y con prueba de regresión. KLOC = miles de líneas de código: 19.9 de PHP, 17.1 de vistas Blade y 19.0 de Kotlin/SQL (55.9 en total), contadas con `git ls-files` y `wc -l`.

Tabla 15

Indicadores del negocio antes y después de ProLech

Indicador	Proceso manual	Con ProLech (verificado)	Evidencia
Acopios duplicados al reintentar el envío	Estimado 5 %	0	Pruebas de reintento con el mismo UUID
Tiempo para liquidar 20 ganaderos	1 a 2 días	< 1 minuto	Botón «Generar liquidaciones»
Tiempo para la planilla de 5 choferes	½ día	< 1 minuto con boleta	Botón «Generar planilla» (Figura 17)
Costo de la leche en las recetas	Fijo: S/ 1.80	Real: S/ 1.67 (30 días)	Tarjeta «Costo real por litro» (Figura 18)
Detección de 2 adulteraciones en la semana	Al cierre de la semana	Inmediata	8 pruebas de `Regla2StrikesTest`
Operaciones de campo sin señal	Imposible	100 % en modo avión	Emulador y 12 pruebas de `SyncManagerTest`

Nota. Datos de la planta cargados con el seeder de demostración (22 ganaderos y 11 932 L en los últimos 30 días). Los valores del proceso manual son estimaciones del equipo.

Tabla 16

Síntesis de las retrospectivas (formato Empezar – Dejar – Continuar)

Sprint	Empezar	Dejar	Continuar
1	Escribir los criterios en Gherkin antes de programar	Repetir reglas de calidad en varios controladores	Migraciones pequeñas, una por cambio

Sprint	Empezar	Dejar	Continuar
2	Probar la integración app–web de punta a punta antes de la Review	Comprometer historias que dependen de pagos externos sin plan B	Una prueba automatizada por cada defecto corregido

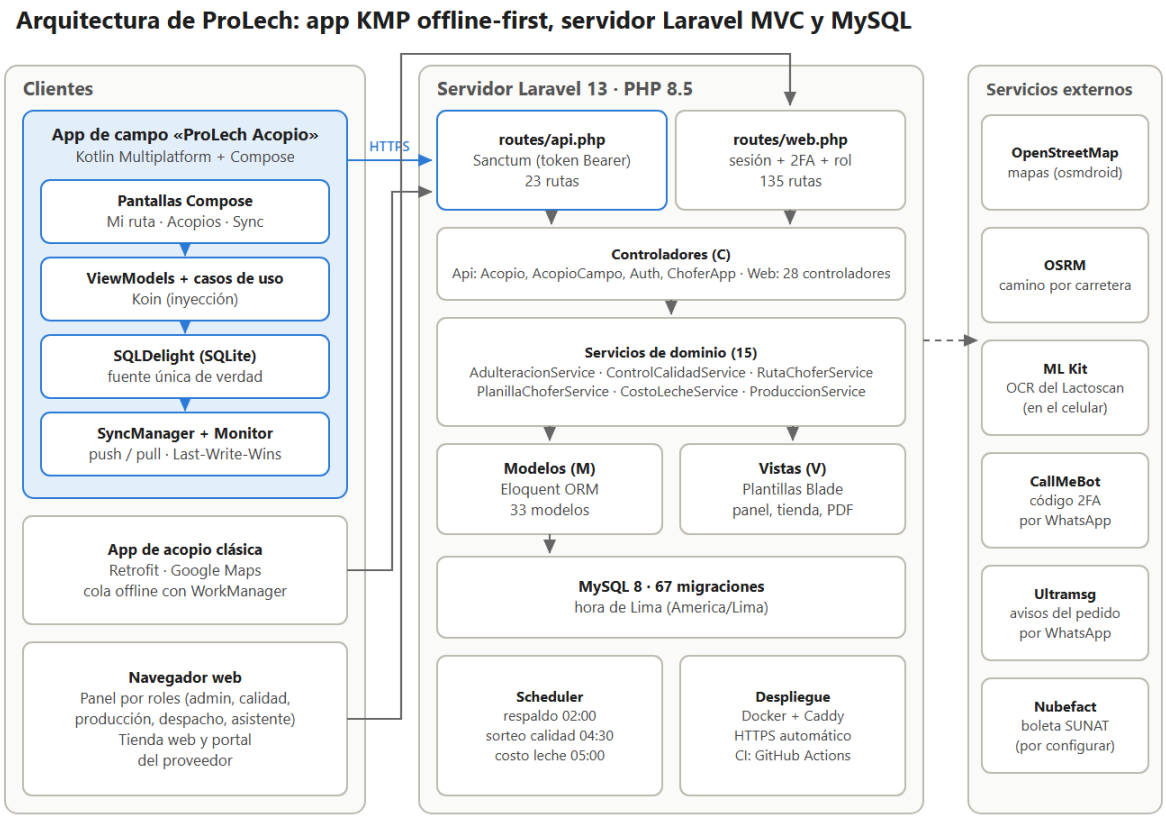
Nota. El detalle con responsables y fechas está en el Apéndice B.

Capítulo III. Arquitectura, Datos y Reglas del Negocio

ProLech tiene un solo servidor y una sola base de datos para todos los canales: la app de campo, el panel web por roles, la tienda en línea y el portal del proveedor. La Figura 7 resume los componentes.

Figura 7

Arquitectura general de ProLech



Nota. La interfaz de la app de campo solo lee y escribe en SQLDelight; el `SyncManager` envía y recibe los cambios con Ktor cuando hay internet. Laravel atiende la API (token Sanctum) y el panel web (sesión, 2FA y rol) sobre los mismos modelos y servicios. Elaboración propia.

3.1 Stack tecnológico

Tabla 17

Stack tecnológico de ProLech

Capa	Tecnología y versión	Uso en ProLech
Lenguaje de la app	Kotlin 2.4 · Kotlin Multiplatform	Lógica compartida (dominio, datos y presentación) para Android; iOS preparado

Capa	Tecnología y versión	Uso en ProLech
Interfaz de la app	Compose Multiplatform 1.11	Pantallas «Mi ruta», «Acopios», «Cuenta» y diálogos
Base de datos local	SQLDelight 2.1.0 (SQLite) · migraciones 1.sqm a 4.sqm	Fuente única de verdad, cola de cambios pendientes, caché de la ruta y GPS
Cliente HTTP	Ktor Client 3.1.3 + kotlinx.serialization	Consumo de la API REST con token Bearer
Inyección de dependencias	Koin 4.0.4	ViewModels, casos de uso, repositorios
Mapas	OSMDroid 6.1.20 + OSRM	Mapa OpenStreetMap sin Google, camino por carretera y paradas
Empaquetado Android	AGP 9.3.3 · R8 · `pe.prolech.acopio`	APK debug y release firmado
Backend	PHP 8.5 + Laravel 13.31 (MVC + servicios)	33 modelos, 32 controladores, 15 servicios, 158 rutas
API y seguridad	Laravel Sanctum 4 · 2FA por WhatsApp	Token para la app; sesión con 2FA y permisos por sección en el panel
Base de datos del servidor	MySQL 8 (67 migraciones)	Datos centrales de la planta
Documentos y gráficos	DomPDF 3.1 · XLSX propio · Chart.js	Boletas, sobres de pago, exportaciones y dashboards
Despliegue	Docker, Caddy (HTTPS automático), GitHub Actions	Servidor de producción y pruebas en cada envío

Nota. Versiones vigentes al cierre del Sprint 2 (28/10/2026), tomadas de `composer.json` y `gradle/libs.versions.toml`.

3.2 Backend Laravel con patrón MVC y capa de servicios

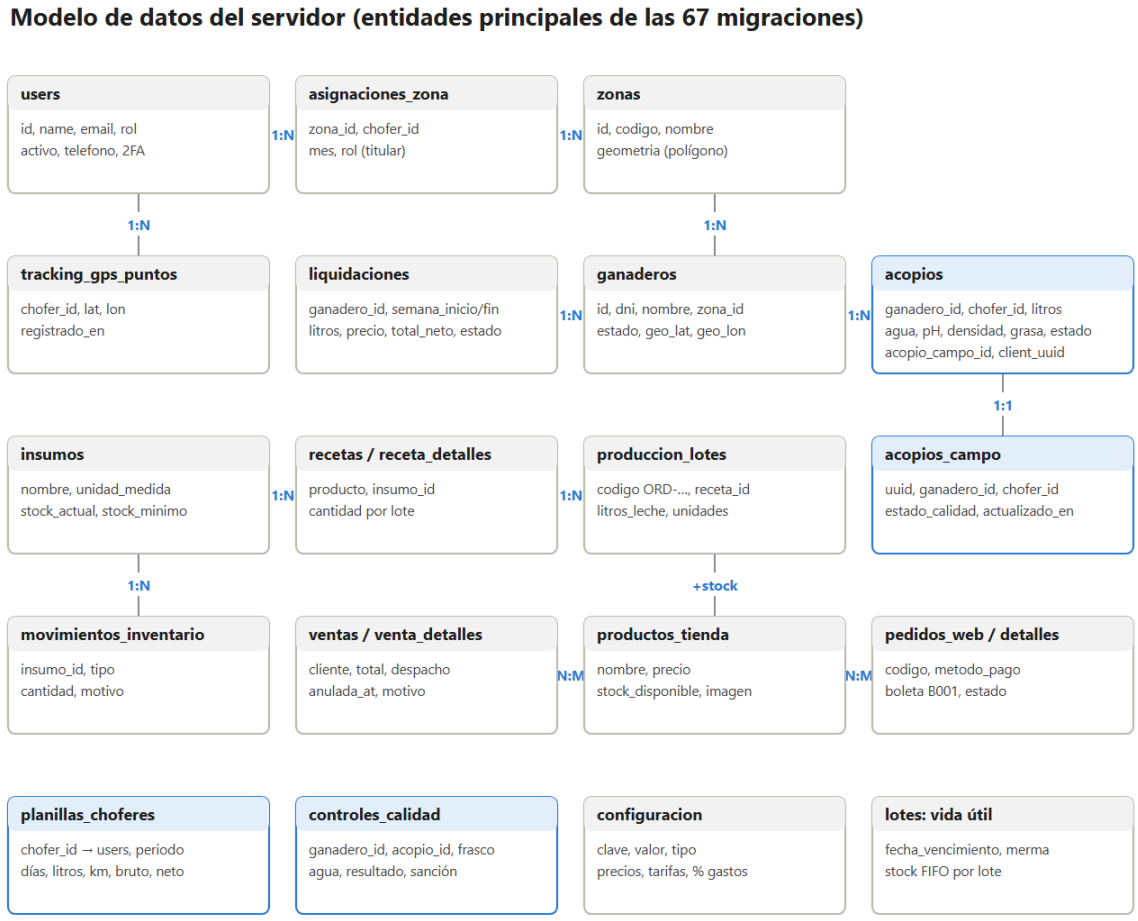
Laravel separa el **Modelo** (clases Eloquent en `app/Models`), la **Vista** (plantillas Blade en `resources/views`) y el **Controlador** (`app/Http/Controllers`, que valida la petición y responde con una vista o con JSON). Para no duplicar reglas entre la API y el panel, la lógica del negocio vive en **servicios** (`app/Services`): por ejemplo, la regla de dos adulteraciones está una sola vez en `AdulteracionService` y la usan la API de la app y el panel. Las rutas del panel pasan por los middleware `auth`, `2fa`, `rol` y `seccion`; las de la API, por `auth:sanctum`.

```
prolech-backend/
├── app/Console/Commands/   # prolech:respaldo, prolech:calidad, prolech:costo-leche,
│                           # prolech:retirar-vencidos, prolech:alertas-stock
├── app/Http/Controllers/Api/ # Auth, AcopioApi, AcopioCampo, ChoferApp (23 rutas)
├── app/Http/Controllers/Web/ # 28 controladores: panel, tienda y portal (135 rutas)
├── app/Models/             # 33 modelos (Acopio, Liquidacion, PlanillaChofer...)
├── app/Services/           # 15 servicios: AdulteracionService, ControlCalidadService,
│                           # PlanillaChoferService, CostoLecheService, StockLotes...
├── database/migrations/    # 67 migraciones versionadas
├── database/seeders/       # ProlechDemoSeeder: Huata con 4 rutas y 5 choferes
├── resources/views/        # admin/, tienda/, proveedor/ (Blade)
├── routes/                 # api.php, web.php, console.php (9 tareas programadas)
└── tests/                  # 30 clases PHPUnit (175 pruebas)
```

3.3 Modelo de datos

Figura 8

Diagrama entidad-relación de la base de datos del servidor



Nota. Se muestran 19 de las tablas creadas por las 67 migraciones. En azul, las tablas que alimenta la app de campo y las del Sprint 2 (planilla y control sorpresa). Elaboración propia.

Tabla 18

Diccionario de datos (extracto de las tablas del negocio)

Tabla	Columna	Tipo	Descripción
acopios	ganadero_id, chofer_id	FK	Quién entregó y quién recogió
	litros, porcentaje_agua, densidad, grasa	decimal	Lectura del Lactoscan en la planta
	estado / tiene_descuento	enum/ bool	aprobado, observado, rechazado; descuento por agua

Tabla	Columna	Tipo	Descripción
	observacion, muestra_codigo, control_resultado	texto	Observación del chofer, frasco sorpresa y su resultado
	client_uuid / acopio_campo_id	uuid / FK	Idempotencia y enlace con la app KMP
liquidaciones	semana_inicio, semana_fin, fecha_pago	date	Jueves a miércoles; pago el viernes
	litros_aprobados, precio_litro, subtotal_normal	decimal	Base del pago
	litros_adulterados, total_bruto, total_neto, estado	decimal / enum	borrador, confirmada, pagada
planillas_choferes	chofer_id, tipo, periodo_inicio, periodo_fin	FK / date	Semanal (jueves–miércoles) o mensual; único por chofer y periodo
	dias_trabajados, litros_recolectados, km_recorridos	int / decimal	Calculados de acopios y GPS
	tarifa_base_dia, monto_base, bono_por_litro, monto_bono	decimal	Jornal y bono
	descuento_pension, adelantos, total_bruto, total_neto	decimal	Descuentos y neto
	detalle_dias (json), estado, metodo_pago, referencia_pago	json / texto	Detalle por día y pago
controles_calidad	tipo, ganadero_id, acopio_id, frasco_codigo	enum / FK	predio, muestra o tanque
	agua, densidad, grasa, ph, resultado, sancion	decimal / enum	normal, leve, grave y sanción aplicada
ganaderos	zona_id, estado, entregas_bloqueadas	FK / enum / int	Activo o suspendido; entregas bloqueadas por sanción
recetas / receta_detalles	rendimiento_esperado_unidades, dias_vida_util / cantidad_requerida	int / decimal	Maestro–detalle de la receta
produccion_lotes	codigo ORD, receta_id, unidades, costo_total, fecha_vencimiento	texto / decimal / date	Lote producido con su costo y vida útil
configuracion	clave, valor, tipo	texto	precio_litro_normal, tarifa_chofer_dia, gastos_generales_pct …

Nota. Tipos lógicos; las migraciones definen las precisiones exactas (por ejemplo `decimal(12,2)` para litros). Elaboración propia.

3.4 Fórmulas del negocio

Las fórmulas se implementaron en servicios con pruebas que comprueban los montos exactos. Los parámetros (precios, tarifas y porcentajes) se cambian en Configuración sin tocar código.

3.4.1 Liquidación semanal del ganadero

Figura 9

Fórmula de la liquidación semanal (LiquidacionController::calcularLiquidacion)

```
P   = precio por litro vigente (normal S/ 1.70 · lluvias S/ 1.20)
PL  = P − descuento_litro_leve (S/ 0.10)        PP = precio de penalidad (S/ 0.65)
L   = litros de la semana (jueves a miércoles), sin los de muestra sorpresa grave


Si hay alguna entrega con agua > 5 % (grave):      Total = L × PP
Si hay 2 o más entregas leves (agua ≤ 5 %):        Total = (L − L_leves) × PL
Si hay 1 entrega leve:                             Total = L × PL
Si no hay incidencias:                             Total = (L_aprob − L_desc) × P + L_desc × PL


Ejemplo real (semana 15/10–21/10): 300.90 L con una entrega de 7 % de agua
  Total = 300.90 × 0.65 = S/ 195.59
```

3.4.2 Regla de 2 adulteraciones y control sorpresa

Un acopio **observado** o **rechazado** cuenta como adulteración. Si el ganadero acumula **dos** en la misma semana de liquidación, `AdulteracionService` lo suspende y avisa al administrador. Además, cada madrugada (04:30) `prolech:calidad sortear` elige al azar los controles del día (por defecto 3: uno en el predio y el resto con frasco que llena el chofer) y uno de tanque por camión; los ganaderos con semáforo rojo o amarillo tienen más probabilidad de salir. El puntaje de riesgo suma 2 puntos por cada observación del chofer y más por resultados previos. Si el resultado es **grave**, esa leche no se paga y se bloquean 3 entregas; si reincide, 6; a la tercera vez, suspensión. Si es **leve**, se aplica descuento en esa entrega y se avisa al ganadero.

3.4.3 Planilla de choferes

Figura 10

Fórmula de la planilla de choferes (PlanillaChoferService::calcular)

```
días      = días del periodo con al menos un acopio no rechazado del chofer
litros    = Σ litros de esos acopios
km(día)   = GPS del celular (tramos ≤ 3 km)  →  si no hay, 2 × largo de la ruta trazada
            →  si no hay, km_ruta_default (45 km)


Base      = días × tarifa_chofer_dia (S/ 60.00)
Bono      = litros × bono_chofer_litro (S/ 0.02)
Bruto     = Base + Bono
Descuento = Bruto × pension_chofer_pct (13 %) + adelantos
Neto      = Bruto − Descuento
Combustible (lo paga la planta, no el chofer) = km × costo_combustible_km (S/ 0.55)


Ejemplo real: Carlos Ramos, semana 22/10–28/10: 6 días, 898.7 L, 191.06 km
  Base = 6 × 60 = 360.00   Bono = 898.7 × 0.02 = 17.97   Bruto = 377.97
  Pensión = 377.97 × 13 % = 49.14   Neto = S/ 328.83   Combustible = S/ 105.08
```

3.4.4 Costo real por litro puesto en planta

Figura 11

Fórmula del costo real por litro (CostoLecheService::calcular)

```
Ventana: últimos 30 días (se recalcula a las 05:00 y al pagar planillas)


Base       = Σ total pagado a ganaderos ÷ Σ litros recibidos en esas semanas
Logística  = (combustible de rutas + pago bruto a choferes) ÷ litros acopiados
Gastos     = (Base + Logística) × gastos_generales_pct (4 %)
Costo_Real_Litro = Base + Logística + Gastos


Ejemplo de referencia:   1.80 + 0.14 + 0.08            = S/ 2.02 por litro
Datos reales (29/09–28/10, 11 931.8 L):
  Base = 1.1635  (semanas con penalidades por adulteración)
  Logística = (1 422.83 + 3 898.64) ÷ 11 931.8 = 0.4460
  Gastos = (1.1635 + 0.4460) × 4 % = 0.0644
  Costo real = S/ 1.6739 por litro
```

El costo real se guarda en el insumo «Leche Cruda» (`CostoLecheService::aplicarAInsumo`) y `Insumo::costoVigente()` lo entrega a las recetas y a los lotes. Así, el costo unitario de cada receta y su **precio sugerido** se actualizan solos:

Figura 12

Costeo dinámico de recetas y precio sugerido (Receta::costoUnitario y precioSugerido)

```
Costo unitario = Σ (cantidad_requerida × costoVigente del insumo) ÷ rendimiento
Precio sugerido = redondeo hacia arriba a S/ 0.50 de [ costo unitario × (1 + margen) ]
Margen real    = (precio de venta − costo unitario) ÷ precio de venta

Ejemplo: Manjar Blanco 1 kg = 4 L de leche × 1.6739 + azúcar, cultivo, vainilla y bolsa
  Costo unitario = S/ 9.30   Sugerido (30 %) = ⌈9.30 × 1.30⌉0.50 = S/ 12.50
  En tienda a S/ 12.00 → margen real 22.5 %
```

3.4.5 Vida útil y stock por lotes

Cada lote guarda `fecha_vencimiento = fecha de producción + dias_vida_util` de su receta. El stock que se puede publicar en la tienda es lo producido y no vendido (`StockLotes::disponibleProducido`), descontado en orden FIFO; cada mañana (06:35) `prolech:retirar-vencidos` retira lo vencido como merma y el panel avisa lo que vence en los próximos días.

3.5 API REST

Tabla 19

Endpoints principales de la API (prefijo /api)

Método y ruta	Función	Respuesta
POST /login	Valida correo y contraseña (máx. 5 intentos por minuto)	Token Sanctum, rol y roles
GET /mi-ruta	Ruta del mes, paradas ordenadas por el trazado, bloqueos y frasco del día	Ruta, trazado, km y paradas
GET /acopios-campo	Pull: acopios del chofer en el servidor	Lista con `uuid` y `actualizado_en`
POST /acopios-campo	Push de creaciones (upsert por UUID); rechaza bloqueados (403)	201 creado o 200 si ya existía
PUT /acopios-campo/{uuid}	Push de ediciones con Last-Write-Wins	Versión vigente
DELETE /acopios-campo/{uuid}	Push de eliminaciones	204, o 404 si ya estaba borrado
POST /acopios-campo/{uuid}/foto	Foto de evidencia (multipart)	Ruta de la foto
POST /tracking/lote	Puntos GPS guardados sin señal (idempotente)	Puntos aceptados
POST /logout	Revoca el token	La siguiente llamada recibe 401

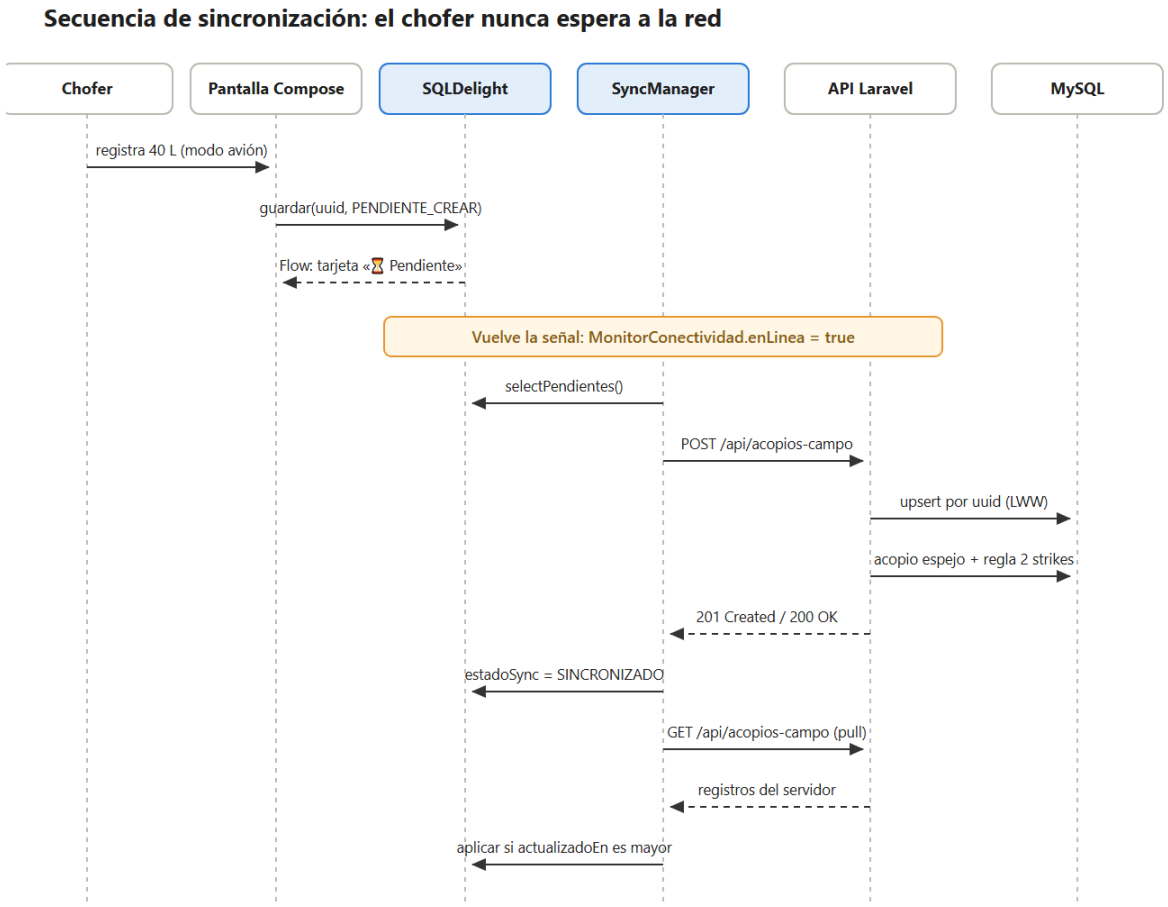
Nota. La API tiene 23 rutas; todas, salvo el inicio de sesión, requieren el token Bearer. Elaboración propia.

3.6 Algoritmo de sincronización offline-first

1. Toda operación del chofer se guarda en SQLDelight con un UUID nuevo y `estadoSync = PENDIENTE_CREAR` (o `PENDIENTE_ACTUALIZAR` / `PENDIENTE_ELIMINAR`); la pantalla nunca espera al servidor.
2. La interfaz observa la tabla local como un `Flow`: el acopio aparece al instante con el aviso «Pendiente».
3. `MonitorConectividad` (NetworkCallback en Android) publica `enLinea`; cuando pasa a verdadero, el `SyncManager` sincroniza solo.
4. Push: se envían los pendientes por `POST`/`PUT` (Ktor) con el UUID y `actualizadoEn`; las eliminaciones por `DELETE` (un 404 se toma como ya borrado).
5. El servidor hace upsert por UUID, aplica Last-Write-Wins, crea el acopio espejo (`AcopioCampoEnlace`), evalúa la regla de 2 strikes y marca el frasco sorpresa como tomado.
6. Pull: se descargan los acopios del servidor y se aplica cada uno solo si su `actualizadoEn` es mayor que el local.
7. Ante un fallo de red el registro queda pendiente y se reintenta; un 4xx definitivo se marca con su motivo; un 401 lleva al inicio de sesión sin borrar lo pendiente.

Figura 13

Diagrama de secuencia de la sincronización offline →online



Nota. Las flechas punteadas son respuestas. El chofer nunca espera a la red. Elaboración propia.

3.7 Arquitectura de la app KMP

La app (`composeApp`) sigue una arquitectura limpia en tres capas: **presentación** (pantallas Compose y ViewModels con `StateFlow`), **dominio** (modelos como `Acopio`, `GanaderoCampo`, `Observacion` y casos de uso) y **datos** (repositorio offline, fuentes locales SQLDelight y remotas Ktor). Koin inyecta las dependencias. El esquema local evoluciona con migraciones versionadas; la 4.sqm (Sprint 2) quitó la medición de calidad al chofer y agregó la observación, el frasco y el bloqueo:

Figura 14

Esquema SQLDelight 2 de la app y migración 4.sqm

```
-- AcopioEntity: fuente única de verdad de la app (id = UUID generado en el celular)
CREATE TABLE AcopioEntity (
    id TEXT NOT NULL PRIMARY KEY,  ganaderoId INTEGER NOT NULL,  ganaderoNombre TEXT NOT NULL,
    litros REAL NOT NULL,  porcentajeAgua REAL NOT NULL,  densidad REAL NOT NULL,
    estadoCalidad TEXT NOT NULL,  actualizadoEn INTEGER NOT NULL,    -- ms: Last-Write-Wins
    estadoSync TEXT NOT NULL DEFAULT 'SINCRONIZADO',  errorSync TEXT,
    grasa REAL,  fotoLocal TEXT,  fotoSubida INTEGER NOT NULL DEFAULT 0,
    observacion TEXT,  observacionNota TEXT,  muestraCodigo TEXT
);

-- 4.sqm (Sprint 2): el chofer ya no mide calidad
ALTER TABLE AcopioEntity ADD COLUMN observacion TEXT;
ALTER TABLE AcopioEntity ADD COLUMN observacionNota TEXT;
ALTER TABLE AcopioEntity ADD COLUMN muestraCodigo TEXT;
ALTER TABLE GanaderoEntity ADD COLUMN bloqueado INTEGER NOT NULL DEFAULT 0;
ALTER TABLE GanaderoEntity ADD COLUMN muestra TEXT;
```

3.8 Interfaces del sistema

Figura 15

Pantallas de la app de campo en el emulador

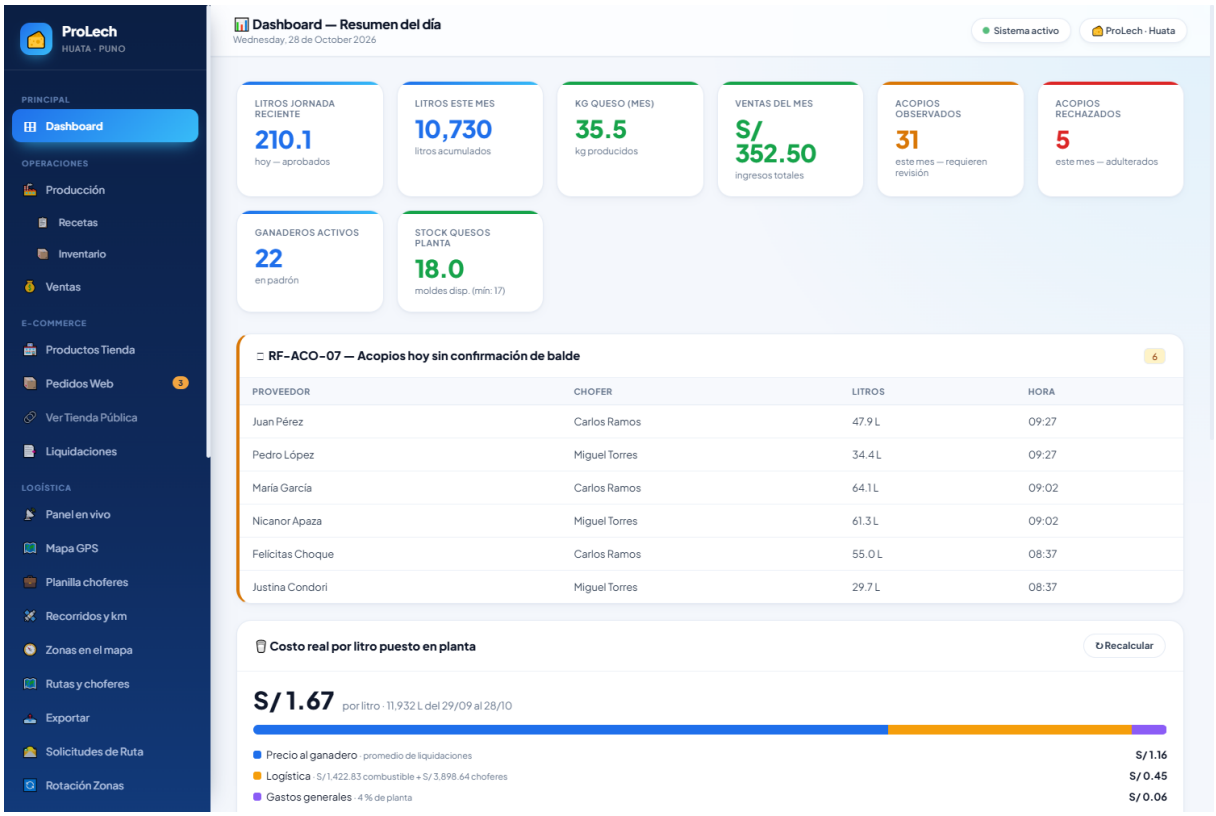


a) Inicio de sesión — b) Mi ruta y mapa — c) Acopio sin señal — d) Sincronizado

Nota. Capturas reales del APK en un emulador Android 14 conectado al servidor Laravel: (a) inicio de sesión con servidor configurable, (b) ruta del día con mapa OSMDroid, (c) acopio guardado en modo avión, (d) acopio sincronizado.

Figura 16

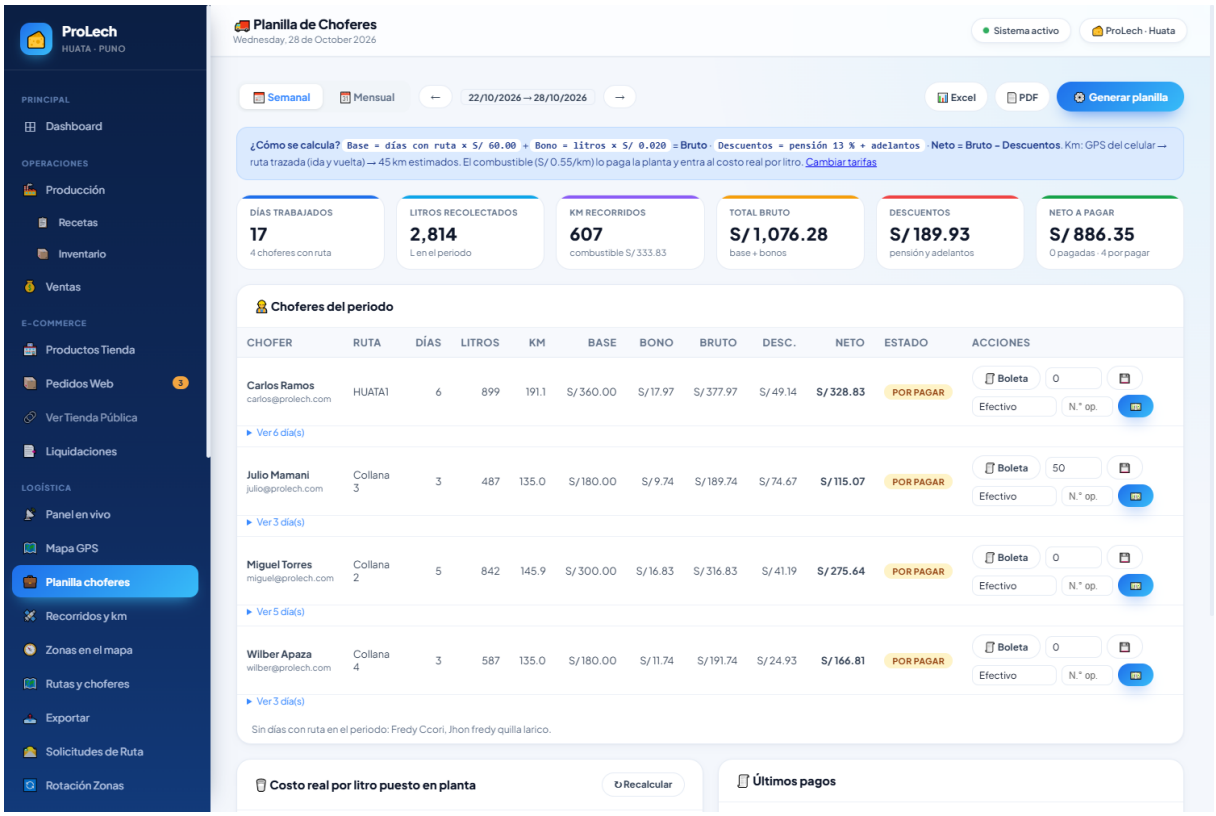
Panel web: dashboard del administrador



Nota. Captura real del 28/10/2026 con los datos de demostración de Huata.

Figura 17

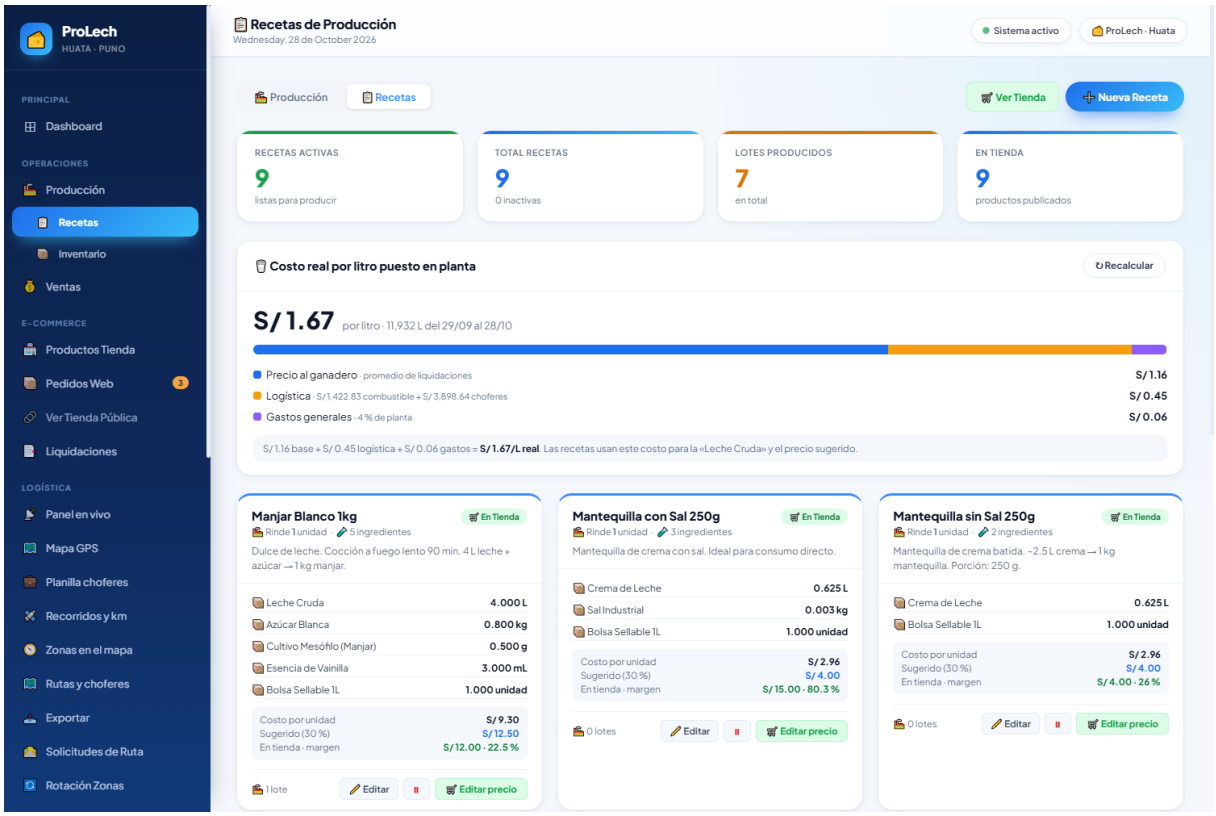
Panel web: planilla de choferes de la semana



Nota. Captura real: 4 choferes con ruta, 2 814 L, 607 km y S/ 886.35 netos por pagar. Cada fila tiene boleta PDF, adelanto y confirmación de pago; abajo, el costo real por litro y los últimos pagos.

Figura 18

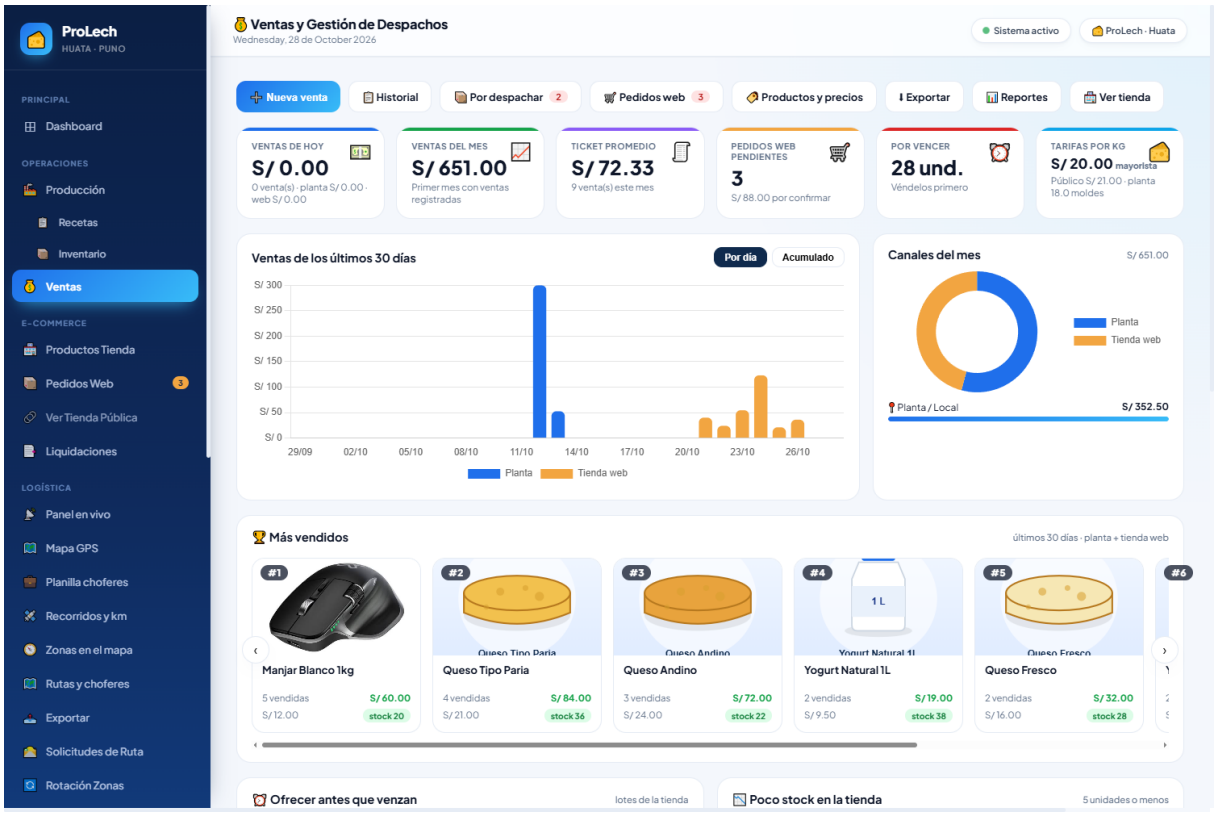
Panel web: recetas con el costo real por litro



Nota. El costo de la «Leche Cruda» de cada receta es el costo real por litro (S/ 1.67): base S/ 1.16 + logística S/ 0.45 + gastos S/ 0.06.

Figura 19

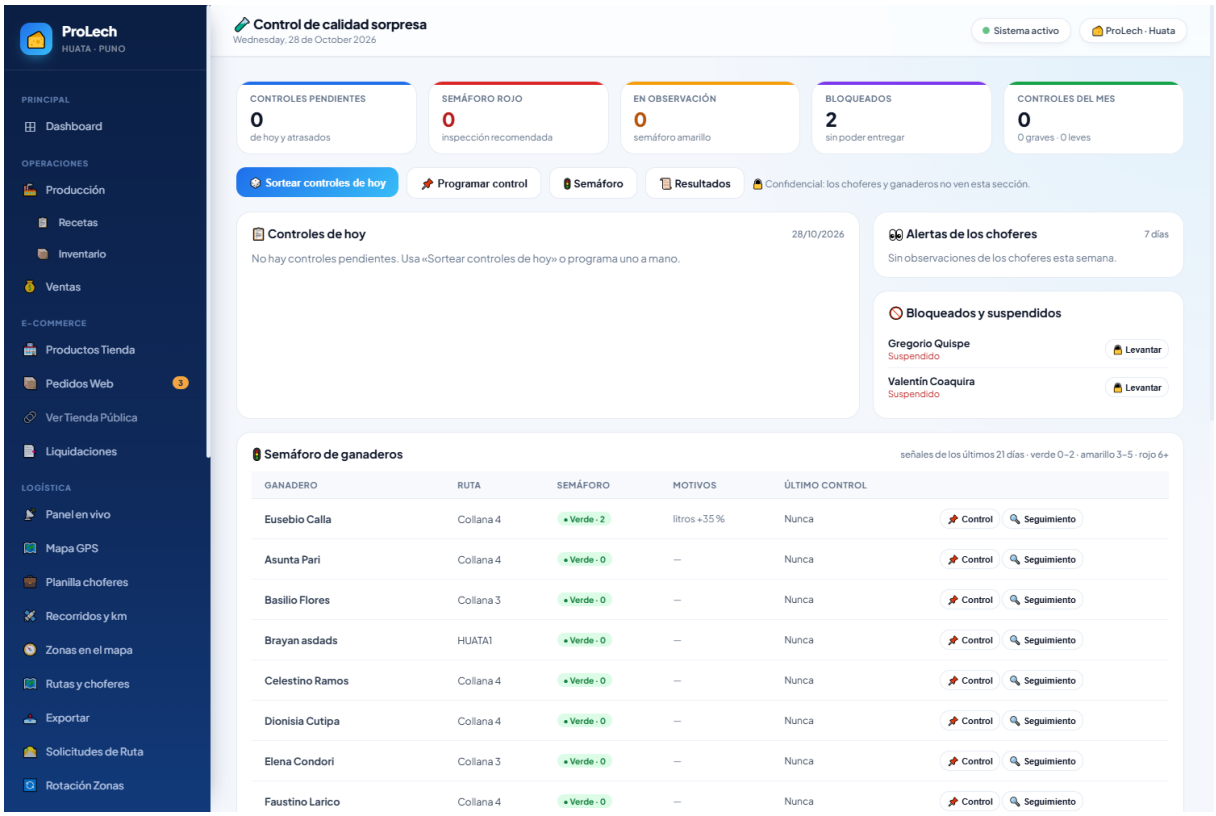
Panel web: dashboard de ventas



Nota. Acciones rápidas, indicadores, ventas de 30 días, carruseles de más vendidos, por vencer y stock bajo.

Figura 20

Panel web: control de calidad sorpresa



Nota. Controles del día, semáforo de riesgo de los ganaderos, alertas de observaciones del chofer y bloqueados.

Figura 21

Tienda en línea (e-commerce)



Nota. Catálogo público con productos publicados desde las recetas y stock tomado de los lotes producidos.

Capítulo IV. Pruebas, Configuración y Adaptación a OpenUP

4.1 Resultados de pruebas

Se ejecutaron 325 verificaciones y todas fueron aprobadas en la versión final del Sprint 2. Las pruebas del backend usan PHPUnit sobre SQLite en memoria; las de la app, kotlin.test con un SQLite real en memoria; las de extremo a extremo se hicieron contra el servidor Laravel con MySQL y los datos de demostración, y la de aceptación, con el APK en un emulador Android 14 y en un celular real.

Tabla 20

Resumen de casos de prueba por tipo

Tipo de prueba	Ejecutados	Aprobados	Fallidos
Unitarias del backend (semana de liquidación, configuración)	9	9	0
API de la app (acopio, campo, enlace, fotos, GPS, rutas, 2 strikes, solicitudes)	59	59	0
Calidad sorpresa, roles múltiples y seguridad	15	15	0
Producción, recetas, precios, vencimiento, planilla y costo por litro	27	27	0
Ventas (carrito, comprobante, anulación, dashboard)	10	10	0
Tienda web (checkout, boleta, seguimiento, Nubefact simulado)	15	15	0
Panel, operaciones, trazado de rutas, seeder y tareas programadas	40	40	0
App KMP: commonTest (SQLDelight, SyncManager, ViewModel, rutas, GPS, trazado)	63	63	0
Extremo a extremo: peticiones al servidor real	70	70	0
Extremo a extremo: código Kotlin de la app contra Laravel	12	12	0
Aceptación en emulador Android 14 y celular Samsung	5	5	0
Total	**325**	**325**	**0**

Nota. Resultados del 28/10/2026 (175 pruebas y 775 aserciones del backend; 63 de la app).

Tabla 21

Casos de prueba representativos

ID	Escenario	Resultado esperado	Resultado
CP-03	Registrar un acopio en modo avión	Se guarda como pendiente y se sube solo al volver la señal	Aprobado
CP-07	Editar el mismo acopio en dos equipos	Gana la versión más reciente	Aprobado
CP-11	Reenviar el mismo acopio (mismo UUID)	Sin duplicados ni segundo strike	Aprobado

ID	Escenario	Resultado esperado	Resultado
CP-15	Segundo acopio observado en la semana	Ganadero suspendido	Fallido en la 1.ª ronda (semana equivocada de domingo a miércoles); corregido
CP-19	Acopio de la app de campo en el panel	Aparece en dashboard y panel en vivo	Fallido (no se veía); corregido con `AcopioCampoEnlace`
CP-41	Liquidar la semana que cierra un miércoles	Incluye los acopios del miércoles	Fallido (SQLite excluía el último día con `whereBetween`); corregido con `whereDate`
CP-44	Planilla: 2 días, 1 000 L, pensión 10 %, adelanto S/ 20	Bruto 140.00, neto 106.00	Aprobado
CP-45	Regenerar una planilla ya pagada	No cambia	Aprobado
CP-46	Costo real con 1 000 L, S/ 180 de logística y 4 %	S/ 2.0592 por litro; receta de 10 L = S/ 20.59	Aprobado
CP-47	Promedio pagado con penalidades	Divide entre litros recibidos	Fallido (sumaba dos veces los litros adulterados); corregido
CP-48	Registrar entrega de un ganadero bloqueado	La app no deja elegirlo y la API responde 403	Aprobado

Nota. CP = caso de prueba. Los fallidos se volvieron a ejecutar tras la corrección y hoy pasan. Elaboración propia.

4.2 Evidencias de pruebas

Backend: planilla de choferes (`tests/Feature/PlanillaYCostoLecheTest.php`)

```
public function test_calcula_dias_litros_km_bono_descuentos_y_neto(): void
{
    [$desde, $hasta] = $this->acopios();     // 2 días × 500 L + 80 L rechazados
    $c = app(PlanillaChoferService::class)->calcular($this->chofer, $desde, $hasta, 20);


    $this->assertSame(2, $c['dias_trabajados']);
    $this->assertEquals(1000, $c['litros_recolectados']);   // el rechazado no cuenta
    $this->assertEquals(140, $c['total_bruto']);            // 2 × 60 + 1000 × 0.02
    $this->assertEquals(106, $c['total_neto']);             // − 10 % − S/ 20
}
```

Figura 22

Salida de consola de las pruebas del backend (cierre del Sprint 2)

```
$ php artisan test
   PASS  Tests\Feature\Regla2StrikesTest
  ✓ primer adulterado desde el campo avisa al admin  0.11s
  ✓ segundo adulterado desde el campo suspende al ganadero  0.13s
  ✓ cuenta juntas las adulteraciones de ambas apps  0.12s
  ✓ aprobados no cuentan  0.11s
  ✓ reintento del celular no cuenta doble  0.12s
  ✓ editar de aprobado a adulterado cuenta como strike  0.14s
  ✓ ganadero ya suspendido no genera avisos repetidos  0.11s
  ✓ app clasica sigue suspendiendo al segundo strike  0.11s
  ...
   PASS  Tests\Feature\PlanillaYCostoLecheTest
  ✓ calcula dias litros km bono descuentos y neto  0.12s
  ✓ generar pagar boleta y exportar desde el panel  1.05s
  ✓ costo real por litro y receta dinamica  0.20s
  ...
  Tests:    175 passed (775 assertions)
  Duration: 38.89s
```

Nota. Salida real del 28/10/2026, recortada: `Regla2StrikesTest` (Sprint 1), `PlanillaYCostoLecheTest` (Sprint 2) y el resumen final.

Figura 23

Resultado de Gradle de las pruebas de la app KMP

```
$ ./gradlew :composeApp:testAndroidHostTest

Clase de prueba               Pruebas  Fallas
AcopioLocalSqlDelightTest           6       0
AcopioViewModelTest                 9       0
CatalogoYSesionTest                 4       0
MejorasCampoTest                   10       0
NivelCargaTest                      1       0
RutaYGpsTest                       15       0
SesionVencidaHttpTest               2       0
SyncManagerTest                    12       0
TrazadoRutaTest                     4       0
Total                              63       0

BUILD SUCCESSFUL
```

Nota. Incluye las pruebas nuevas del Sprint 2: ganadero bloqueado por calidad y registro con observación y frasco de muestra.

4.3 Gestión de la configuración

El proyecto usa dos repositorios Git: `prolech-backend` (Laravel) y `aplicacionnnnnn` (app Kotlin). Cada cambio se guarda en un commit en español que indica el área y qué se hizo. Los secretos no entran al repositorio (`.env`, `local.properties`, `keystore.properties` y la llave `prolech-release.jks` están en `.gitignore`). El flujo es: rama `feature/HU-xx`, pruebas primero, suite completa en local, CI en GitHub Actions, revisión de otro integrante e integración al cierre del sprint.

Tabla 22

Incrementos entregados por sprint

Incremento	Sprint	Contenido principal
v0.5	1	Acopio con Lactoscan, calidad, regla de 2 strikes, ganaderos, rutas, rotación, liquidación con sobre de pago, calidad en planta, reportes, 2FA
v1.0	2	App KMP offline-first con SQLDelight y Ktor, mapa OSMDroid y GPS, panel en vivo, producción maestro–detalle con vencimientos, ventas y tienda, control sorpresa
v1.1	2	Planilla de choferes y costo real por litro con costeo dinámico de recetas

Nota. Elaboración propia a partir del historial de Git.

Figura 24

Últimos commits del repositorio del backend

```
$ git log --oneline -8
eb34a23 Costo real por litro: precio promedio sobre los litros recibidos en las semanas liquidadas
0861f17 Planilla de choferes y costo real por litro puesto en planta
0dcc25a Control de calidad sorpresa: semáforo, sorteo, frascos, resultados y bloqueo por entregas
2496267 Ventas: dashboard completo con acciones, indicadores, gráficos y carruseles
f88c378 Producción: vencimiento por lote y stock de la tienda desde lo producido
2adf272 Precios: costo por unidad desde la receta, precio sugerido y margen
87143fa Detalles: usuarios editables, ruta actual real, caminos en el mapa GPS y avisos al chofer
aa1d9e1 Usuarios con varios roles: rol principal + accesos adicionales
```

Nota. Salida real del repositorio `prolech-backend` al 28/10/2026.

4.4 Comparación con el ciclo de vida de OpenUP

OpenUP organiza el proyecto en cuatro fases con iteraciones internas (Kroll y MacIsaac, 2006). El equipo trabajó con Scrum en dos sprints y los relacionó con esas fases:

Tabla 23

Correspondencia entre los sprints de Scrum y las fases de OpenUP

Fase OpenUP	Objetivo de la fase	Equivalente en el proyecto	Entregable producido
Inicio	Acordar el alcance y la visión	Sprint 0 (agosto): entrevistas en la planta y Product Backlog con MoSCoW	Visión y requerimientos (Capítulo I)
Elaboración	Estabilizar la arquitectura	Primera mitad del Sprint 1: modelo de datos, API con token y servicios	Arquitectura MVC + servicios (Capítulo III)
Construcción	Completar la funcionalidad	Resto del Sprint 1 y Sprint 2	Incrementos v0.5, v1.0 y v1.1 con sus pruebas
Transición	Entregar al usuario	Última semana del Sprint 2: APK, manual, respaldos y despliegue preparado	Manual de usuario, APK y guía de despliegue

Nota. Elaboración propia.

Tabla 24

Diferencias principales entre Scrum y OpenUP en el proyecto

Aspecto	Scrum (aplicado)	OpenUP (adaptado)
Roles	Product Owner, Scrum Master, Developers	Stakeholder, jefe de proyecto, analista, arquitecto, desarrollador, tester
Planificación	Sprint Backlog por sprint de 4 semanas	Plan por fases y plan de iteración
Documentación	Historias con Gherkin, INVEST y MoSCoW	Visión, casos de uso, arquitectura y plan de pruebas
Gestión del riesgo	Implícita en la priorización del backlog	Explícita: la Elaboración ataca primero el riesgo técnico

Nota. Elaboración propia.

El mayor riesgo técnico, la sincronización sin señal, se abordó en el Sprint 2. La comparación con OpenUP muestra que, de haberse prototipado en la Elaboración (Sprint 1), se habría evitado el retrabajo de unir los acopios de la app con la tabla que lee el panel.

Conclusiones

1. ProLech permite registrar el acopio sin señal: la app KMP guarda cada entrega en SQLDelight y la sincroniza sola con Ktor; no se generó ningún duplicado gracias al UUID creado en el celular y a la regla Last-Write-Wins.
2. El backend Laravel con MVC y una capa de servicios atiende al panel, a la tienda y a la app sobre los mismos modelos, de modo que reglas como la de dos adulteraciones o el control sorpresa se escriben una sola vez.
3. La planilla de choferes y el costo real por litro convierten datos operativos (días, litros, km, pagos) en información económica: con los datos de demostración, el litro puesto en planta cuesta S/ 1.67 y las recetas toman ese costo para su precio sugerido.
4. Scrum en dos sprints de cuatro semanas permitió entregar 128 de 139 puntos (92.1 %) con una velocidad promedio de 64 puntos; todas las historias Must se completaron y lo pendiente depende de servicios externos.
5. Las pruebas automatizadas (175 del backend y 63 de la app) y las de extremo a extremo detectaron 43 defectos, como el último día excluido de la semana en SQLite o el doble conteo de litros adulterados en el costo, todos corregidos con prueba de regresión.

Recomendaciones

1. Contratar un VPS y un dominio y publicar con la configuración de `deploy/` (Docker + Caddy con HTTPS automático) (HU-25).
2. Crear la cuenta de Nubefact para que la boleta tenga validez ante SUNAT (HU-26).
3. Completar la pestaña «Inspecciones» de la app para que el inspector registre los controles en el predio sin abrir el panel (HU-27).
4. Validar con contabilidad las tarifas de la planilla (jornal, bono y aporte de pensión) y el porcentaje de gastos generales antes de usar el costo real para fijar precios.
5. Copiar cada semana un respaldo y la llave de firma del APK fuera del servidor.

Referencias

American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7.ª ed.). https://doi.org/10.1037/0000165-000

Cash App. (s.f.). *SQLDelight documentation*. https://sqldelight.github.io/sqldelight/

Cohn, M. (2004). *User stories applied: For agile software development*. Addison-Wesley.

JetBrains. (s.f.). *Kotlin Multiplatform*. Kotlin Documentation. https://kotlinlang.org/docs/multiplatform.html

JetBrains. (s.f.). *Ktor client*. Ktor Documentation. https://ktor.io/docs/client-create-new-application.html

Kroll, P., y MacIsaac, B. (2006). *Agility and discipline made easy: Practices from OpenUP and RUP*. Addison-Wesley Professional.

Laravel. (2026). *Laravel documentation* (versión 13.x). https://laravel.com/docs

OSMDroid. (s.f.). *osmdroid: OpenStreetMap-Tools for Android*. https://github.com/osmdroid/osmdroid

Schwaber, K., y Sutherland, J. (2020). *La guía de Scrum: La guía definitiva de Scrum: Las reglas del juego*. Scrum.org, https://scrumguides.org

SmartBear. (s.f.). *Gherkin reference*. Cucumber Documentation. https://cucumber.io/docs/gherkin/reference/

Washizaki, H. (Ed.). (2024). *Guide to the software engineering body of knowledge (SWEBOK Guide)* (versión 4.0). IEEE Computer Society. https://www.swebok.org

Apéndices

Apéndice A Manual de Usuario de ProLech

A 1 Requisitos. Celular Android 7.0 o superior con GPS. Para el panel web: Chrome, Edge o Firefox actualizado. La primera sesión de cada chofer requiere internet para descargar su ruta.

A 2 Instalación de «ProLech Acopio».

1. Copie el APK que entrega la planta y ábralo; si Android lo pide, permita instalar apps de origen desconocido.
2. En el inicio de sesión toque «Servidor» y escriba la dirección que indique el administrador.
3. Ingrese su correo y contraseña. Acepte los permisos de ubicación y cámara.

Tabla A1

Tareas por usuario

Usuario	Tarea	Pasos
Chofer	Ver la ruta del día	Mi ruta →paradas en el orden del camino, km y capacidad del camión
Chofer	Registrar una entrega	Acopios →ganadero →litros → ¿cómo se ve la leche? →Registrar (si aparece «Hoy toca muestra», llenar el frasco indicado)
Chofer	Cuenta	
Calidad	Control sorpresa	Panel →Control sorpresa →controles del día →registrar resultado (agua, densidad ..). →el sistema aplica la sanción
Asistente	Liquidar a los ganaderos	Panel →Liquidaciones →Generar →Confirmar →Pagar →sobre PDF
Administrador	Pagar a los choferes	Panel →Planilla choferes →Semanal o Mensual →Generar planilla
Administrador	Costo real por litro	Panel →Recetas (o Producción o Dashboard) →tarjeta «Costo real por litro» →↻ Recalcular; tarifas en Configuración
Jefe de producción	Orden de producción	Panel →Producción →agregar productos →Guardar (descuenta insumos, crea lotes con vencimiento)
Despacho	Registrar una venta	Panel →Ventas →Nueva venta →carrito →Guardar → Comprobante
Cliente	Comprar en la tienda	Tienda →carrito →datos →pago →boleta →seguimiento con código

Nota. Elaboración propia.

Apéndice B Actas de los eventos Scrum

B.1 Sprint 1 – Core de acopio, calidad y liquidación

Tabla B1

Acta de Sprint Planning del Sprint 1

Campo	Contenido
Fecha y duración	Jueves 03/09/2026, 16:00 a 20:00 (4 h), laboratorio de la EP de Ingeniería de Sistemas
Asistentes	Alex Brayan Gutiérrez Huanca (PO), Jhon Saul Mamani Cruz (SM), Jayson Smit Coaquira Ramírez (Developer), Jhon Fredy Quilla Larico (Developer)
Sprint Goal	El chofer registra cada entrega con su calidad Lactoscan, la planta castiga la adulteración con la regla de 2 strikes, rota a sus choferes y liquida a sus ganaderos cada semana
Capacidad	4 integrantes × 20 días × 3 h = 240 h. Sin velocidad histórica: se comprometen 63 puntos
Historias comprometidas	HU-01 a HU-12 (63 puntos): 8 Must, 3 Should, 2 Could
¿Cómo se hará? (parte 2)	Laravel 13 con servicios; primero modelo de datos y API con token; la regla de 2 strikes en un solo servicio
Riesgos	Definir con la planta la semana de liquidación (jueves–miércoles) y los rangos de agua; costo de mensajes 2FA
Acuerdos	Gherkin antes de programar; una migración por cambio; demo en el celular en la Review

Nota. Elaboración propia.

Tabla B2

Registro de Daily Scrum del Sprint 1 (extracto)

Fecha	Integrante	¿Qué hice ayer?	¿Qué haré hoy?	Impedimentos
08/09	Jayson Smit Coaquira Ramírez	Modelo Acopio y migraciones de calidad	API POST /acopios con client_uuid	Ninguno
08/09	Jhon Fredy Quilla Larico	CRUD de ganaderos	Zonas con polígono en el mapa	Leaflet no dibuja en modales
08/09	Jhon Saul Mamani Cruz	Tablero y casos de prueba HU-05	Casos de prueba HU-06 y HU-07	Ninguno
08/09	Alex Brayan Gutiérrez Huanca	Rangos de agua con la planta	Gherkin de HU-08 (liquidación)	Falta precio de temporada de lluvias
17/09	Jayson Smit Coaquira Ramírez	AdulteracionService	Semana de liquidación jueves–miércoles	El domingo calcula la semana anterior (bug)
17/09	Jhon Fredy Quilla Larico	Exportación XLSX propia	Plan inopinado de calidad	Ninguno
24/09	Jayson Smit Coaquira Ramírez	Sobre de pago PDF	Estados confirmada y pagada	Ninguno
24/09	Jhon Fredy Quilla Larico	Comunicados y reuniones	Portal del proveedor	No alcanza el tiempo para HU-12

Nota. Daily de 15 min de pie, de lunes a viernes a las 08:00.

Tabla B3

Acta de Sprint Review del Sprint 1

Campo	Contenido
Fecha y duración	Miércoles 30/09/2026, 2 h
Asistentes	Equipo Scrum y administrador de la planta (stakeholder)
Incremento presentado (v0.5)	Acopio con Lactoscan y calidad, regla de 2 strikes, ganaderos y rutas en el mapa, rotación mensual, liquidación con sobre de pago, calidad en planta, reportes y 2FA
Historias aceptadas	HU-01 a HU-11 (60 de 63 puntos)
No terminadas	HU-12 Portal del proveedor (3 puntos): vuelve al Product Backlog y entra al Sprint 2
Comentarios del stakeholder	«Necesitamos que el chofer pueda registrar sin señal» y «queremos saber cuánto nos cuesta de verdad el litro»
Cambios al Product Backlog	Se priorizan HU-13 y HU-14 como Must; se agregan HU-22 (planilla) y HU-23 (costo real por litro)
Conformidad	Alex Brayan Gutiérrez Huanca (Product Owner)

Nota. Elaboración propia.

Tabla B4

Retrospectiva del Sprint 1 (Empezar – Dejar – Continuar)

Empezar	Dejar	Continuar
Escribir los escenarios Gherkin antes de programar (responsable: PO)	Repetir reglas de calidad en varios controladores	Migraciones pequeñas, una por cambio
Probar la semana de liquidación con fechas de borde: domingo y miércoles (SM)	Dejar las pruebas manuales para el último día	Servicios con transacciones para el dinero

Nota. Acciones con responsable revisadas en el Sprint Planning del Sprint 2.

B.2 Sprint 2 – App offline-first y negocio

Tabla B5

Acta de Sprint Planning del Sprint 2

Campo	Contenido
Fecha y duración	Jueves 01/10/2026, 16:00 a 20:00 (4 h)
Asistentes	Alex Brayan Gutiérrez Huanca (PO), Jhon Saul Mamani Cruz (SM), Jayson Smit Coaquira Ramírez (Developer), Jhon Fredy Quilla Larico (Developer)
Sprint Goal	El chofer trabaja sin señal con la app KMP y GPS; la planta paga la planilla de choferes, conoce el costo real de su litro, produce con recetas maestro–detalle y vende en línea
Capacidad y velocidad	240 h; velocidad del Sprint 1: 60 puntos. Se comprometen 79 puntos (19 de ellos Should/Could con dependencias externas)
Historias comprometidas	HU-12 (arrastre) y HU-13 a HU-27
¿Cómo se hará? (parte 2)	Kotlin Multiplatform con SQLDelight 2 como fuente única de verdad y Ktor; planilla y costo como servicios con parámetros en Configuración

Campo	Contenido
Riesgos	KMP con AGP 9; mapas sin Google (OSMDroid); contratar VPS y Nubefact depende de pagos
Acuerdos	Toda historia de la app se prueba en modo avión; las fórmulas de dinero se validan con la planta con un ejemplo real

Nota. Elaboración propia.

Tabla B6

Registro de Daily Scrum del Sprint 2 (extracto)

Fecha	Integrante	¿Qué hice ayer?	¿Qué haré hoy?	Impedimentos
06/10	Jayson Smit Coaquira Ramírez	Esquema SQLDelight y repositorio offline	SyncManager con Ktor (push)	Gradle no genera SQLDelight con AGP 9
06/10	Jhon Fredy Quilla Larico	Carrito de ventas	Comprobante imprimible	Ninguno
06/10	Jhon Saul Mamani Cruz	Casos de prueba modo avión	Prueba en emulador Android 14	Ninguno
15/10	Jayson Smit Coaquira Ramírez	Mapa OSMDroid y camino OSRM	GPS en lote	Mapa «Access blocked» con paquete com.example
15/10	Jhon Saul Mamani Cruz	Pruebas E2E de la API	Celular real en la red local	Resuelto: applicationId pe.prolech.acopio
22/10	Jayson Smit Coaquira Ramírez	Control sorpresa (servicio y panel)	App: observación y frasco	Ninguno
27/10	Jayson Smit Coaquira Ramírez	PlanillaChoferService y vistas	CostoLecheService y recetas dinámicas	Ninguno
27/10	Alex Brayan Gutiérrez Huanca	Validó tarifas de chofer con la planta	Validar %de gastos generales	Falta dato de luz y agua de la planta
28/10	Jhon Fredy Quilla Larico	Exportación Excel de la planilla	Preparar demo de la Review	VPS y Nubefact sin contratar

Nota. Daily de 15 min, de lunes a viernes a las 08:00.

Tabla B7

Acta de Sprint Review del Sprint 2

Campo	Contenido
Fecha y duración	Miércoles 28/10/2026, 2 h
Asistentes	Equipo Scrum y administrador de la planta (stakeholder)
Incremento presentado (v1.1)	App KMP offline-first con sincronización, mapa y GPS; panel en vivo; producción maestro–detalle con vencimientos; ventas con dashboard; tienda en línea; control sorpresa; planilla de choferes con boleta; costo real por litro en recetas
Demostración	Acopio en modo avión en el celular y sincronización al volver la red; generación y pago de la planilla con boleta PDF; recálculo del costo real (S/ 1.67/L) y precio sugerido del manjar blanco
Historias aceptadas	HU-12 a HU-24 (68 de 79 puntos)

Campo	Contenido
No terminadas	HU-25 VPS y dominio, HU-26 Nubefact, HU-27 pestaña Inspecciones (11 puntos)
Comentarios del stakeholder	Revisar con contabilidad el aporte de pensión; agregar más adelante el pago a acopiadores eventuales
Conformidad	Alex Brayan Gutiérrez Huanca (Product Owner)

Nota. Elaboración propia.

Tabla B8

Retrospectiva del Sprint 2 (Empezar – Dejar – Continuar)

Empezar	Dejar	Continuar
Probar la integración app–web de punta a punta antes de la Review (SM)	Comprometer historias que dependen de pagos externos sin plan B	Una prueba automatizada por cada defecto corregido
Validar las fórmulas de dinero con un ejemplo real de la planta antes de programarlas (PO)	Guardar la llave de firma del APK solo en una PC	Pruebas en modo avión y en celular real
Subir los repositorios a GitHub para activar la CI (Dev)	Usar `whereBetween` con fechas en SQLite	Parámetros de negocio en Configuración, no en el código

Nota. Elaboración propia.

Apéndice C Plantillas de Gestión de la Configuración

Figura C1

Plantilla de solicitud de cambio (pull request)

```
## [Área]: [resumen corto del cambio]
**Historia:** HU-[xx] · **Sprint:** [1|2] · **Revisor:** [integrante]
### ¿Qué cambia?
- [cambio 1]
### ¿Cómo probarlo?
1. php artisan test   /   ./gradlew :composeApp:testAndroidHostTest
2. [paso manual, por ejemplo: registrar un acopio en modo avión]
### Checklist (Definición de Hecho)
- [ ] Escenarios Gherkin cumplidos   - [ ] Suite completa en verde
- [ ] Migración probada en MySQL y SQLite   - [ ] Funciona en modo avión
```

Figura C2

Reporte de un defecto real del Sprint 2

```
## BUG-47: el costo real sumaba dos veces los litros adulterados
**Caso de prueba:** CP-47 · **Severidad:** Media · **Versión:** v1.1 · **Módulo:** Costos
**Resultado esperado:** precio promedio = total pagado ÷ litros recibidos.
**Resultado obtenido:** S/ 1.08 (denominador = aprobados + adulterados; en semanas
  con penalidad grave los litros ya estaban en aprobados).
**Corrección:** dividir entre los litros no rechazados de las semanas liquidadas → S/ 1.16.
```

Apéndice D Evidencias de Despliegue

Tabla D1

Datos del entorno de pruebas y de producción

Elemento	Valor
Servidor de pruebas	Laragon en Windows 10: PHP 8.5, MySQL 8.0.30, `php artisan serve --host=0.0.0.0 --port=8000`
Base de datos	MySQL 8, 67 migraciones, zona horaria America/Lima, respaldo diario a las 02:00 (se conservan 14)
Datos de demostración	`ProlechDemoSeeder`: 4 rutas, 5 choferes, 22 ganaderos, acopios, liquidaciones, 12 planillas de choferes (8 pagadas), recetas y tienda
Tareas programadas	respaldo 02:00 · limpieza 03:00–03:20 · sorteo de calidad 04:30 · costo leche 05:00 · alertas de stock 06:30 · vencidos 06:35 · bloqueos 23:30
Aplicación móvil	ProLech Acopio (`pe.prolech.acopio`), Android 7.0 o superior; probada en emulador Android 14 y Samsung S21 FE
Producción (preparado)	Docker (PHP-FPM 8.5 + MySQL 8.4) con Caddy y certificado Let's Encrypt automático
Dominio y VPS	Pendiente de contratar (HU-25)

Nota. Datos reales al 28/10/2026.

Figura D1

Migración, datos de demostración y costo real en el entorno de pruebas

```
$ php artisan migrate
  2026_10_28_100001_planilla_choferes_y_costo_leche ..... 568.26ms DONE
$ php artisan db:seed --class=ProlechDemoSeeder
  ✓ProlechDemoSeeder: Huata lista para la demostración.
$ php artisan prolech:costo-leche
  Costo real por litro: S/ 1.6739
```

Nota. Salida real del 28/10/2026.